A series of test-driven small coding puzzles:

HOW TO CODE A database in 45 steps with Go

Day 05: KV store
Day 10: Tables
Day 20: SQL
Day 30: LSM-Tree
Day 40: ACID compliant, relational DB

trialofcode.org

Contents

Code a database in 45 steps

This series of **test-driven small coding puzzles** lets you code a database **from scratch** (no dependencies). Through this project, you can:

- Learn **database internals** and computer science basics.
- **Level up your skills** through challenging work.
- Build an impressive **personal project** for your resume.

Project content

The project implements several key parts:

- KV storage engine.
- SQL and relational databases.
- Indexes and data structures.

The scope is wide, but it is split into many tiny steps. **Each step is simple, with at most a few dozen lines of code**. You will see how complex ideas grow from simple ones. In this sense, you invent a database from zero.

Prerequisites

- Have learned any programming language.
- Can use the command line (Linux or Windows).

Go language

- Go is easy to learn, so no prior experience is needed.
- You may use any language, since the core ideas are language-agnostic.
- It can also be used as a practice project for learning other languages.

Starter code and tests

- Download: https://trialofcode.org/database/db_project+solution.zip

Each step is in a separate directory. For step 1, enter `db_project/0001` and run the tests:

```
go test .
```

The build will fail. You need to write the missing functions. If you have problems:

- Check the test cases to gain insights.
- Refer to the author's solution in the `db_solution` directory.
- Discuss with the author by email: hi@trialofcode.org

Other projects like this

This is first **Trial of Code** project, I plan to release more puzzles that lead to complex, advanced software. Check my website and subscribe for updates:

https://trialofcode.org

Go Quick Start

Familiarize yourself with the major features of Go. Skip if you already know Go. You are not required to use Go for this project, though.

Syntax

Variables

```
// Declare variable types
var a int       // default is 0
var b string    // default is ""
// Assign values
a = 123
b = "hi"
// Declare and initialize
var c int =123
d := "hi"       // Type is inferred automatically
// Assign multiple variables
a, c = c, a
```

An uninitialized value is guaranteed to be a **zero value** (`0`, `false`, `""`, `nil`, etc).

Statements

```
// If-else has no parentheses
if a != c {
    // ...
} else {
    // ...
}
// Switch does not fall through like C
switch a {
case 123:
    // ...
case 456:
    // ...
default:
    // ...
}
```

```
// Declare and use a variable in if
if err := f(); err !=nil {
    // ...
}
```

```
// Loops
for i :=0; i < N; i++ {
    // ...
}
```

```
for zzz {
    // ...
}
for {
    if xxx {
        break
    }
    if yyy {
        continue
    }
}
```

Functions

```
func f(a int, b string) error {
    return nil
}
// Multiple return values
func g() (string, bool) {
    return "hi", false
}
// Assign multiple return values
s, b := g()
// Named return values
func h() (s string, b bool) {
    s = "hi"
    b = false
    return
}
```

```
// If a function returns an error, put error last
func foo() (string, error) {
    // ...
}

result, err := foo()
if err != nil {
    // handle error
} else {
    // use result
}
```

Data Types

Numeric Types

```
// Fixed size
bool, byte
int8, int16, int32, int64
uint8, uint16, uint32, uint64
float32, float64
// Size depends on system: 32-bit or 64-bit
int, uint
```

```
// Type conversion
a := 1  // int
var  b int64
b = a           // compile error
b = int64(a)    // convert
```

- Integer overflow and conversion follow **two's complement**, no UB like C.
- Type conversion must be explicit.

Pointers/References

```
a := 1
var  ptr *int    // declare pointer
ptr = &a         // take address
b := *ptr        // dereference
```

Go has GC, so pointer use is restricted, which includes arbitrary pointer arithmetic or storing pointers as other types. You won't hit these issues if you avoid `unsafe`.

struct, method

```
type C struct  {
    foo string,
    bar int,
}
// Initialize struct
c := C{foo: "hi", bar: 123}
// Access fields
c.bar = 456
bar1 := c.bar
// Pointer
pc := &c
bar2 := (*pc).bar   // dereference before using field
bar3 := pc.bar      // no need explicit dereference
```

```
// Normal function
func  foo1(c *C) {
    c.foo = "hi"
}
// Method with pointer receiver
func  (c *C) foo2() {
    c.foo = "hi"
}
// Method with value receiver
func  (c C) bar() {
    println(c.bar)
}
// Call function
foo1(&c)
// Call methods
c.foo2()    // pointer
c.bar()     // value
```

Fixed-Size Arrays

```
var  a [2]byte       // 2 elements, init to 0
var  b [2][4]byte    // 2 elements, each is [4]byte
// Assign
b = [2][4]byte{{0, 0, 0, 0}, {0, 0, 0, 0}}
b[1][3] = 123
```

Strings

A string is just a some bytes without text encoding. It can convert to/from **`[]byte`**.

```
type string struct {
    ptr *byte   // data
    len int     // length
}
```

```
s := "asdf"     // {ptr: "asdf", len: 4}
len(s)          // 4
// Strings are immutable
s[0] = 'b'      // compile error
// Slice without copying
s[2:4]          // {ptr: "df", len: 2}
// Concatenation creates a new string
s = "hi"  + s
```

slice

In Go, slice actually has 2 unrelated uses shared by the same representation:

a. A view of an array, with a pointer and length.
b. A growable dynamic array, like C++ vector or Python list.

```
type sliceT struct {
    ptr *T
    len int
    cap int
}
```

- **`len()`** and **`cap()`** return length and capacity.
- A **`nil`** slice is **`{ptr: nil, len: 0, cap: 0}`**.

These 2 uses are often confused, so they are explained separately.

a. Slice as an array view

```
// An array
var  arr [8]byte  = [8]byte{0, 1, 2, 3, 4, 5, 6, 7}
// Slice an array
s1 := arr[1:5]      // {ptr: &arr[1], len: 4, cap: 7}
// Iterate
for  i :=0; i < len(s1); i++ {
    println(s1[i])  // 1 2 3 4
}
```

```
// Use `range` instead
for i := range s1 {
    println(s1[i])
}
for i, v := range s1 {
    println(v)
}
// Out of bounds causes panic
v := s1[4]

// Reslice, returns a new slice
s2 := s1[2:3]       // {ptr: &arr[3], len: 1, cap: 5}
for _, v := range s2 {
    println(v)      // 3
}
```

Bounds checks:

- `s[i]`: `i < len`
- `s[i:j]`: `j <= cap`

Usually, reslicing shrinks the `len`, although expansion is also allowed. **When a slice is an array view, only `len` matters, `cap` can be ignored.**

```
// shorthands
s[:]    // s[0:len(s)]
s[:j]   // s[0:j]
s[i:]   // s[i:len(s)]
```

b. Slice as a dynamic array

```
// Allocate array of len 2
arr1 := make([]byte, 2)     // {ptr: 0x100, len: 2, cap: 2}
// Allocate array with cap 2, len 0
arr2 := make([]byte, 0, 2)  // {ptr: 0x200, len: 0, cap: 2}
// Append elements
arr2 = append(arr2, 123)    // {ptr: 0x200, len: 1, cap: 2}
arr2 = append(arr2, 123)    // {ptr: 0x200, len: 2, cap: 2}
// Reallocate when cap is exceeded
arr2 = append(arr2, 123)    // {ptr: 0x300, len: 3, cap: 8}
// Append multiple elements
arr2 = append(arr2, 2, 3)   // {ptr: 0x300, len: 5, cap: 8}
```

Only dynamic slices can be appended. **A slice that is an array view must not be appended**, since it would modify the array.

Common slice operations

```
copy(dst, src)                  // copy data
s1 = slices.Clone(s)            // allocate and copy
s = slices.Concat(s1, s2, s2)   // concatenate
s = slices.Delete(s, i, j)      // delete s[i:j]
s = slices.Insert(s, i, e1, e2) // insert at i
idx = slices.Index(s, v)        // find index
slices.Sort(s)                  // sort in place
```

A slice as an **array view** is only a reference and must not be appended. A slice as a **dynamic array** owns the array and can do any operation. A dynamic slice can be used as an array view, but not the reverse.

interface

An interface is a set of method rules. Any type that implements these methods can be used as the interface.

```
type Reader interface {
    Read(p []byte) (n int, err error)
}
// Type that implements the interface
type T struct { /* ... */ }
func (s *T) Read(p []byte) (n int, err error) { /* ... */ }
// A function that takes an interface as an argument
func useReader(r Reader) { /* ... */ }

obj := &T{}         // concrete type
useReader(obj)      // converted to interface
```

Calling methods via an interface does not depend on the concrete type, similar to abstract classes in OOP. An interface has 2 pointers:

```
type interfaceFoo struct {
    object  uintptr    // data pointer
    typeInfo uintptr    // type info
}
```

Converting a value to an interface stores the value pointer and its type info. Method calls use the type info to find the implementation.

An uninitialized interface has both pointers set to **`nil`**. This is different from converting a **`nil`** pointer value to an interface.

Recover the original type

- Use **`iface.(T)`** to test and extract type **`T`**.
- **`interface{}`** has 0 methods. Any type can convert to it.

```
var anything interface{}
anything = 1
anything = "asdf"

// Convert back to concrete type
integer, ok := anything.(int)   // ok = false
str, ok := anything.(string)    // ok = true
// Type switch
switch origin := anything.(type) {
case int:
    integer = origin
case string:
    str = origin
}
```

Tests

A test file ends with **_test.go**, e.g., **foo_test.go**. Test functions starts with **Test** and takes a ***testing.T** argument.

```
package foo

import "testing"

func TestSomething(t *testing.T) {
    if 1 +2 !=3 {
        t.Fail()
    }
}
```

Run all test cases with this command: **go test .**

0101: KV Interface

Memory vs. disk

Software called databases comes in many forms. Some are pure in-memory, such as Redis and Memcached. Some are disk-based, such as MySQL and SQLite.

In-memory databases are limited by RAM size, so traditional databases are disk-based. But even disk-based databases often include in-memory data structures. So different types of databases are not unrelated. We can start from an in-memory KV and build the project by adding code step by step.

A database is an application of data structures. Whether data is stored in memory or on disk makes no difference in principle, but many issues appear in implementations. You rarely see these topics in books, but you can learn them in a project.

Key-value vs. table

Databases can be classified by their interfaces: key-value (KV), relational, and other special types. KV is like map or dict in programming languages, with operations like get, set, del. Relational databases use tables (rows and columns), usually operated with SQL.

In terms of features, relational databases seem to do more. However, more complex relational databases are built on simpler KV systems, often called storage engines. For example, LevelDB and RocksDB can be used as standalone KV stores, and also have SQL DBs built on top. So database implementation starts from KV.

OLAP vs. OLTP

Databases can be divided by usage into 2 types: **O**n**l**ine **A**nalytical **P**rocessing and **O**n**l**ine **T**ransaction **P**rocessing. These names are just names; the words "analytical" and "transaction" have no precise meaning. OLAP focuses on analyzing large amounts of data, while OLTP focuses on returning results in real time using indexes. For example:

- OLAP: to count active users, Bob runs a SQL query and `COUNT()`s matching users.
- OLTP: to show user count in real time in an app. But there are many users, an extra counter is maintained.

OLTP needs real-time results, so each operation has an upper bound on resource use (CPU, IO, memory). If you know data structures, this bound is usually $O(\log N)$. An index uses extra information to support real-time queries, trading space for time. In applications, "index" does not always mean a database index feature. In the example above, the app maintains its own extra index because the database has no index for counting.

OLAP workloads may consume large resources, so they are often stored separately from OLTP. Traditional databases like MySQL and PG are OLTP types. After the rise of "big data", databases specialized for OLAP became popular.

Although both are called relational DBs, OLAP usually stores data by column, while OLTP stores by row. Their underlying tech and use cases differ. This project only considers OLTP. Do not mix them up when learning.

In-memory KV

An in-memory KV starts from the simplest map:

```
type KV struct {
    mem map[string][]byte
}

func (kv *KV) Open() error {
    kv.mem = map[string][]byte{} // empty
    return nil
}

func (kv *KV) Close() error { return nil }

func (kv *KV) Get(key []byte) (val []byte, ok bool, err error)
func (kv *KV) Set(key []byte, val []byte) (updated bool, err error)
func (kv *KV) Del(key []byte) (deleted bool, err error)
```

- Keys and values are **`[]byte`**, so they can hold any binary data.
- Since Go maps cannot use **`[]byte`** as keys, string is used.
- Disk IO will be added later, so these interfaces return **`error`**.
- **`Set`** and **`Del`** must report whether the database state changed.

Enter the **`db_project/0101`** directory. Implement **`Get`**, **`Set`**, and **`Del`**. Run tests:

```
go test .
```

ACID

Atomicity, Consistency, Isolation, Durability, the so-called ACID, are often used to describe databases, as if they are 4 DB properties. Many people feel they are hard to understand. This is because they are vague ideas without clear definitions, and often mean different things.

Atomicity literally means indivisible. For example, writing n bytes to a file:

a. With concurrent reads, a reader may see half-written data.
b. If power fails mid-write, after recovery only half the data may exist.

Both can be described as lack of atomicity, and databases must address them. But one is about concurrency, the other about durability. They are unrelated problems.

Consistency is used to describe internal database logic, business logic, or distributed systems, without even a vague definition.

Isolation refers to whether a transaction is affected by other concurrent transactions.

Durability means that once the DB reports success, the data can be trusted not to disappear. In implementation, this cannot be considered alone as atomicity is also involved.

Database behavior is complex, and ACID does not fully describe it. In practice, you must learn the specific behavior of each database. When implementing a database yourself, you will directly face real problems databases must solve, and their solutions.

0102: Binary Serialization

Serialization

To store data types from a programming language on disk or send them over a network, they must be converted into a byte sequence. This is called serialization.

For example, a KV pair:

```
type Entry struct {
    key []byte
    val []byte
}

func (ent *Entry) Encode() []byte
```

Implement **`Encode()`** using the following format:

```
| key size | val size | key data | val data |
| 4 bytes  | 4 bytes  |   ...    |   ...    |
```

For example, key=a and val=bb returns **`[]byte(1, 0, 0, 0, 2, 0, 0, 0, 'a', 'b', 'b')`**.

For slices, strings, and other variable-length types, the size must come first. Here the size is stored as little-endian uint32. Use **`binary.LittleEndian.PutUint32()`** to convert integers into 4 bytes.

```
func (ent *Entry) Encode() []byte {
    data := make([]byte, 4+4+len(ent.key)+len(ent.val))
    binary.LittleEndian.PutUint32(data[0:4], uint32(len(ent.key)))
    binary.LittleEndian.PutUint32(data[4:8], uint32(len(ent.val)))
    copy(data[8:], ent.key)
    copy(data[8+len(ent.key):], ent.val)
    return data
}
```

Deserialization

Deserialization parses a byte sequence back into data. Next, implement:

```
func (ent *Entry) Decode(r io.Reader) error
```

When calling **`Decode()`**, the caller does not know how many bytes are needed, so a slice cannot be passed. Instead, use the standard library **`io.Reader`** interface:

```
type Reader interface {
    Read(p []byte) (n int, err error)
}
```

Inside **`Decode()`**, call **`r.Read()`** to read data, like reading from a file. But since it is an interface, the underlying implementation is not necessarily a file.

Enter the **`db_project/0102`** directory. Implement **`Entry.Decode()`**. Run tests:

```
go test .
```

io.Reader and io.Writer

`io.Reader` is used for input, and the corresponding output interface is **`io.Writer`**.

```
type Writer interface {
    Write(p []byte) (n int, err error)
}
```

Any type that implements **`Read()`** or **`Write()`** can be used as an **`io.Reader`** or **`io.Writer`**. The benefit of these interfaces is flexibility. For example, the parameter of **`Decode()`** is not a concrete type. In the next step, it will read from a log file, while in this step, test cases read from memory. You can check **`bytes.Buffer`** in the test cases to learn how it works.

In fact, **`Encode()`** could also use **`io.Writer`** instead of returning a slice, though it is not necessary.

Unix syscalls **`read`** and **`write`** use a similar design. They can operate on very different resources: files, network sockets, pipes, IPC. The common point is input and output.

Serialization methods

All serialization methods are similar, differing only in details. To serialize variable-length data like strings, the simplest way is to put the length first, then the data. The length is an integer, and there are many ways to encode it. Some formats use 2 bytes, some 4 bytes, some use variable-length varint, and some like Redis use decimal digits.

Besides this **binary format**, there are **text formats** such as JSON and XML. “Binary” has nothing to do with number bases; it is just the opposite of “text”. Most text formats do not encode string length, but use **delimiters** to mark the end of data. JSON uses quotes, XML uses tags.

Text formats look intuitive, but are hard to implement. Because encoded data cannot contain delimiters, text formats require complex **escaping**. Even simple JSON has many bugs across implementations. Compared to simple binary serialization, text formats also waste CPU.

Beyond complexity, text formats often have some arbitrary limits. For example, JSON cannot support arbitrary binary data, so base64 is used, which wastes even more. Many JSON libraries do not support 64-bit integers. So unless necessary, do not use text formats.

For binary serialization, there are implementations like Protobuf and MsgPack, but they are not as widely used as JSON. Many low-level projects invent their own formats. This is because binary serialization is simple and not worth adding a library dependency. Text formats, due to their complexity, are best handled by a library.

0103: Log Storage

Append-only logs

Like text logs, a database **log only appends entries at the end of the file, and never modifies or deletes existing entries**. Log entries record every update to the database. For example, a log with 4 records:

	operation	state
0		{}
1	set k1=x	{k1=x}
2	set k2=y	{k1=x, k2=y}
3	set k1=z	{k1=z, k2=y}
4	del k2	{k1=z}

When the database starts, it reads the log and applies updates in order, producing the final state `{k1=z}`. This implements **incremental updates**. This is one reason to use a log. Other uses will appear later.

You may ask: **how are old log records removed?** This will be solved later: when the log reaches a certain size, it is merged into the main data structure (LSM-Tree or B+Tree).

The log records every **state change**. Even with concurrent transactions, as long as it is not a distributed system, state changes can be serialized in the log. So logs can be used for replication and rollback (undo). However, a log cannot grow forever, so it cannot be the main storage and is only auxiliary. A log is not strictly required; a database can also be built on copy-on-write data structures with log-like behavior.

Log records

From the example above, the log has 2 types of records: update and delete. So a flag is added to **`Entry`** to distinguish them.

```
type Entry struct {
    key     []byte
    val     []byte
    deleted bool
}
```

The serialization format becomes:

```
| key size | val size | deleted | key data | val data |
| 4 bytes  | 4 bytes  | 1 byte  |   ...    |   ...    |
```

Modify these 2 functions:

```
func (ent *Entry) Encode() []byte
func (ent *Entry) Decode(r io.Reader) error
```

Read and write the log file

Struct definition:

```
type Log struct {
    FileName string
    fp       *os.File
}

func (log *Log) Open() (err error) {
    log.fp, err = os.OpenFile(log.FileName, os.O_RDWR|os.O_CREATE,0o644)
    return err
}

func (log *Log) Close() error {
    return log.fp.Close()
}
```

Log IO interfaces:

```
func (log *Log) Write(ent *Entry) error {
    _, err := log.fp.Write(ent.Encode())
    return err
}

func (log *Log) Read(ent *Entry) (eof bool, err error) {
    err = ent.Decode(log.fp)
    if err == io.EOF {
        return true, nil
    } else if err != nil {
        return false, err
    } else {
        return false, nil
    }
}
```

When the database starts, it repeatedly calls **`Log.Read()`** until the end of file. After that, each update calls **`Log.Write()`** to append to the end.

`Log.Read()` detects end of file using **`io.EOF`** (**E**nd **O**f **F**ile). So **`Entry.Decode()`** must propagate errors returned by **`io.Reader`**.

Reading the log

Add this struct to **`KV`**:

```
type KV struct {
    log Log
    mem map[string][]byte
}
```

```
func (kv *KV) Open() error {
    if err := kv.log.Open(); err != nil {
        return err
    }
    kv.mem = map[string][]byte{}
    // TODO
}
```

```
func (kv *KV) Close() error { return kv.log.Close() }
```

Implement **KV.Open()** to load the log into **KV.mem**. Requirement: later entries override earlier ones.

Writing the log

Modify set and del to add log writes:

```
func (kv *KV) Set(key []byte, val []byte) (updated bool, err error)
func (kv *KV) Del(key []byte) (deleted bool, err error)
```

Enter the **db_project/0102** directory. Run tests:

```
go test .
```

Summary

The log persists incremental updates to disk. But this log still has problems. What happens if the database loses power during use? This is the next issue to address.

0104: fsync

Ensuring writes reach disk

Since data is stored on disk, we must ensure it is actually written to disk. If we only write to a file, a power loss can cause the file to disappear or be filled with 0x00. These are problems a database must solve.

A database guarantees that written data is not lost. This is usually called **durability**. The guarantee is defined by a successful return to the caller. If the database crashes before returning, the state is uncertain to the caller. But if the caller receives success, it can trust the write not disappearing.

Cache and fsync

Each file write does not directly map to a disk write. The OS has a memory cache. Writes go to the cache first, then are synced to disk later. This allows merging repeated writes and improves throughput. Repeated reads also benefit.

This cache is called the page cache. The page here matches the CPU virtual memory page. A page is the smallest unit of IO, with a fixed size (usually 4K or 16K). You may ask how disk IO cache relates to virtual memory. Look into `mmap`. Also note that database docs often call B-tree nodes "pages". Do not confuse them.

Besides the page cache, disks may also have their own RAM cache. To ensure data reaches disk, an operation must flush all cache layers and **wait for completion**. This is the `fsync` syscall on Linux. In Go, call it via `Sync()` on `*os.File`.

```
func (log *Log) Write(ent *Entry) error {
    if _, err := log.fp.Write(ent.Encode()); err !=nil {
        return err
    }
    return log.fp.Sync() // fsync
}
```

Windows has a corresponding operation, so `Sync()` also applies.

Some app-level file APIs also have caches that need flushing, such as `fflush()` in C. Go file IO maps directly to OS APIs and has no extra cache.

fsync the parent directory

On Linux, `fsync` ensures file data is written, but does not ensure the file itself exists. This is because a file is recorded by its parent directory. If a directory entry is added (file creation) but not written to disk before power loss, the file cannot be reached, even if its data is on disk. To fix this, call `fsync` on the directory.

That is: creating files, renaming files, and deleting files all require `fsync` on the containing directory.

This is Unix-specific. Windows does not need this. The Go standard library has no method for `fsync`ing a directory, so you must invoke syscalls directly:

```
func createFileSync(file string) (*os.File, error) {
    fp, err := os.OpenFile(file, os.O_RDWR|os.O_CREATE, 0o644)
    if err != nil {
        return nil, err
    }
    if err = syncDir(path.Base(file)); err != nil {
        _ = fp.Close()
        return nil, err
    }
    return fp, err
}

func syncDir(file string) error {
    flags := os.O_RDONLY | syscall.O_DIRECTORY
    dirfd, err := syscall.Open(path.Dir(file), flags, 0o644)
    if err != nil {
        return err
    }
    defer syscall.Close(dirfd)
    return syscall.Fsync(dirfd)
}
```

Replace **`os.OpenFile()`** with this function:

```
func (log *Log) Open() (err error) {
    log.fp, err = createFileSync(log.FileName)
    return err
}
```

Unix basics

`open` opens or creates a file and returns a number on success. This number identifies the file and is used for later operations, such as **`fsync`**. This number is called a file descriptor (fd).

A file descriptor is not limited to files. It can refer to other OS resources, such as network sockets or directories. On Unix, **`open`** can open directories, though the returned fd cannot be read or written like a file. All fds must be closed.

You can read the **`os.File`** source in the standard library. It mainly wraps various syscalls.

Summary

Durability is achieved via **`fsync`**. However, this is still not enough for power loss cases: a log record may be only half written before a crash (atomicity). We'll handle this next.

0105: Checksum

Incomplete Writes

When appending a record to the log, we want it to either be completely written or not written at all. This can be called **atomicity**. Incomplete writes are known as torn writes. However, file writes do not guarantee atomicity in the case of power loss. Not only is the appended data not atomic, but even the file size could be incorrect. For example, appending 1000-bytes record may result in one of the following situations:

a. File size increases by 1000, but data isn't fully written.
b. File size increases by 1000, but no data is written, and the new space is filled with 0s or garbage.
c. File size increases by 500.

In any case, only the last record will be affected, while previously **`fsync`**ed records remain intact. This is another reason to use a log in databases.

Disk Write Atomicity

The CPU provides atomic memory read/write operations that handle data the size of an integer. While CPU atomic operations focus on concurrency, we're considering atomicity for persistence in the case of power loss. At the hardware level, writing a single sector is likely atomic[1]. A sector is the smallest unit of disk read/write, usually 512B or 4K.

Many software systems rely on atomicity at the sector level to achieve larger-scale atomicity. For example, when writing a log, a sector at the beginning of the log can be reserved to store the position of the last log entry. After the log is successfully fsynced, the sector is updated (including **`fsync`**ing this sector). This ensures atomicity for the entire log.

Even without atomicity at the hardware level, software can achieve atomicity without needing 2 **`fsync`**s.

Checksum

Suppose log writes are not atomic, but if we can detect an incomplete write, we can simply ignore it. The last record affected will be the one before the last successful **`fsync`**. A checksum can help here. A checksum is a hash, and different data will almost certainly have different checksums. By storing the checksum for each record, we can identify incomplete writes.

Use the standard library's **`crc32.ChecksumIEEE()`** to compute the checksum for log records and prepend it to the record:

```
| crc32   | key size | val size | deleted | key data | val data |
| 4 bytes | 4 bytes  | 4 bytes  | 1 byte  |   ...    |   ...    |
```

Modify these functions:

```go
func (ent *Entry) Encode() []byte
func (ent *Entry) Decode(r io.Reader) error

var ErrBadSum = errors.New("bad checksum")
```

[1] https://stackoverflow.com/a/61832882/13057409

`Entry.Decode()` should return the appropriate errors:

- `ErrBadSum`: checksum mismatch (data not fully written).
- `io.EOF`: reached the end of the file (all records are valid).
- `io.ErrUnexpectedEOF`: file ended prematurely (file size is incorrect, data not fully written).

io.ReadFull()

The `io.Reader` interface allows returning fewer bytes than the requested `buf` length. For example, requesting 4 bytes might return only 3, but this doesn't mean the end of the data (eof). While this typically won't happen when reading from a file, it's common with sources like TCP sockets. So when using `io.Reader`, you should loop until the returned byte count satisfies the buffer size. The standard library's `io.ReadFull()` can replace this loop:

```
func ReadFull(r Reader, buf []byte) (n int, err error)
```

If the buffer is only partially filled and `eof` is reached, `io.ReadFull()` will return `io.ErrUnexpectedEOF`. This perfectly matches the requirements of the `Decode()` function.

Handling Incomplete Log Records

Check the error returned by `Entry.Decode()` to ignore the last incomplete log record. That is, `eof=true`.

```
func (log *Log) Read(ent *Entry) (eof bool, err error)
```

The test case for this step simulates recovering from the last incomplete log write.

You may wonder: how do we know if a failed checksum is in the last log entry? There is no way to know, because the record's size header might be incorrect.

Checksum Use

In addition to incomplete writes, checksums can also detect data corruption caused by hardware failures, such as flipped bits due to disk or memory faults. This is a rare occurrence with normal hardware. However, **this is not the purpose of checksums in databases**, as databases cannot recover from such silent data losses. If a record in the middle of the log is corrupted, it's impossible to know if subsequent records exist.

Cryptographic hash functions (like sha256 or md5) could also be used as checksums. However, there is no reason to use them, because specialized checksum functions are smaller and faster. Even simple checksum methods, like the 16-bit integer sum used in TCP/IP, are sufficient. However, care must be taken for the case where all bytes are 0. In such cases, the checksum should not be 0, and `crc32` does not have this issue.

Summary

With log + checksum + fsync, the database can recover after a power loss and ensure previously successful writes are not lost. This is the core functionality of a database. This is why SQLite is commonly used for mobile data storage rather than simply writing JSON to a file. It's not just for easier querying but because basic file operations cannot ensure durability and atomicity.

For now, we have only a log. If data structures are added in the future, atomicity and durability (A and D of ACID) must be reconsidered.

0201: Data Types

Union

This step starts using KV to build a relational DB. Like Excel, one DB can hold many tables. A table has rows and columns. Each column can choose a data type, unlike KV which only has string/bytes. We will implement 2 data types: **`int64`** and **`[]byte`**.

```
type CellType uint8

const (
    TypeI64 CellType = 1
    TypeStr CellType = 2
)

type Cell struct {
    Type CellType
    I64  int64
    Str  []byte
}
```

`Cell` represents different data types. Differnciated by **`Type`**, the data is either **`I64`** or **`Str`**. Go has no union type, so some space is wasted.

Serialization

This step implements serialization and deserialization of **`Cell`**:

```
func (cell *Cell) Encode(toAppend []byte) []byte {
    switch cell.Type {
    case TypeI64:
        // TODO
    case TypeStr:
        // TODO
    }
}

func (cell *Cell) Decode(data []byte) (rest []byte, err error)
```

Requirements:

- **`Cell.Encode()`** appends the serialized result to the input slice. When serializing a row next, multiple **`Cell`** values will be joined. The same slice can be reused to avoid repeated memory allocation.
- **`Cell.Decode()`** deserializes from the input slice and returns the remaining data.
- **`int64`** uses **`binary.LittleEndian`**. **`[]byte`** uses this format:

```
| length  | data |
| 4 bytes | ...  |
```

Endianness

A CPU uses fixed-size binary numbers, usually 8, 16, 32, or 64 bits. Bit 0 is the lowest bit. For example, 0b0101 = 4 + 1 = 5. From bit 0 to bit 3, the values are 1, 0, 1, 0. When writing

down a number on paper, the low digits are on the right and the high digits on the left. When writing down an array, element 0 is on the left and higher memory addresses are on the right. This difference leads to **big endian** and **little endian**.

A 32-bit integer in a CPU register is just 32 bits. When stored in memory, it is grouped into 4 bytes. If the bytes follow **natural order** (low bits in byte 0, high bits in byte 3), it is little endian. If they follow **written order** (low bits in byte 3, high bits in byte 0), it is big endian. For example, 0x11223344 in hex: little endian is **`44 33 22 11`**, big endian is **`11 22 33 44`**.

Historically, some computers used big endian. This is why many network protocols like TCP/IP use big endian. Today, little endian is dominant. A little endian CPU must convert big endian data, which adds work. New data formats and protocols mostly use little endian. Big endian still has a special use (sorting) that will appear later.

Signed and Unsigned Integers

You may notice that **`binary.LittleEndian`** only has methods using uint64, and none for int64. This is because signed and unsigned integers can be converted directly:

```
cell.I64 = int64(binary.LittleEndian.Uint64(data[0:8]))
```

You need to understand how negative numbers are encoded. Using int64 as an example:

- Positive numbers $[0,2^{63})$ use the same encoding as uint64.
- Negative numbers $[-2^{63},0)$ match $[2^{63},2^{64})$ in uint64.

Converting between uint64 and int64 means doing nothing. The effect is:

- The uint64 range is split in half. int64 positive numbers and 0 map to the first half. Negative numbers map to the second half.
- For small positive numbers, uint64 and int64 look the same in bits.

The difference between signed and unsigned integers is only how the CPU interprets the bits. Since there is no real conversion, we can freely use type casts.

Sign Bit

The highest bit of a signed integer shows whether it is negative. Many people naively assume computers store numbers as sign + absolute value. That method was only used by some old machines. Modern computers use the method above, called **two's complement**. If sign + absolute value were used, there would be both +0 and −0, which exists only in floating point numbers, not integers.

0202: Table Schema

Structs

Table name, column names, column types, primary key:

```
type Schema struct {
    Table string
    Cols  []Column
    PKey  []int // which columns are the primary key?
}
type Column struct {
    Name string
    Type CellType
}
```

For example, this table:

```
create table `link` (
    `time` int64 not null,
    `src` string not null,
    `dst` string not null,
    primary key (`src`, `dst`)
);
```

Is represented as:

```
    schema := &Schema{
        Table: "link",
        Cols: []Column{
            {Name: "time", Type: TypeI64},
            {Name: "src", Type: TypeStr},
            {Name: "dst", Type: TypeStr},
        },
        PKey: []int{1, 2}, // (src, dst)
    }
```

Primary Key

The primary key is the unique ID of a row. It is made of 1 or more columns. To operate on a row, you must first find it by the primary key. So the primary key is the K in KV, and non-primary-key columns are stored in V. This lets a KV store implement a relational database (OLTP type).

Besides primary keys, indexes are also built with KV. An index lets you find the primary key K, then use it to get V. Like a book: the page number is the primary key, while the "table of contents" and the "index" are indexes. They help you find the page number, then the content. In some cases, an index already has enough data, so V does not need to be read, like reading only the table of contents.

Both primary keys and indexes are KV. The difference is that the primary key is required. So indexes are also called secondary indexes, and the primary key is called the primary key. Some databases, like SQLite, allow tables without a user-defined primary key. This

is because SQLite tables have a hidden auto-generated primary key. User-visible primary keys are just normal indexes, with no real difference between primary and secondary.

Encode a Row as KV

A row in the database is represented as:

```
type Row []Cell

func (schema *Schema) NewRow() Row {
    return make(Row, len(schema.Cols))
}
```

`Schema.NewRow()` returns a row of the right size, not initialized.

Next, implement serialization and deserialization:

```
func (row Row) EncodeKey(schema *Schema) (key []byte)
func (row Row) EncodeVal(schema *Schema) (val []byte)
func (row Row) DecodeKey(schema *Schema, key []byte) (err error)
func (row Row) DecodeVal(schema *Schema, val []byte) (err error)
```

Requirements:

- Input or output rows must match the schema.
- Use **`Cell.Encode()`** and **`Cell.Decode()`** from the previous step.
- Follow the column order defined in the schema.

Key Prefix

A database has many tables, so encoded keys need a prefix to tell tables apart. We'll use the table name:

```
func (row Row) EncodeKey(schema *Schema) (key []byte) {
    key = append([]byte(schema.Table), 0x00)
    // TODO
}
```

A 0x00 separator is added after the table name to avoid key conflicts from name prefixes. For example, table names **`ab`** and **`abc`** have prefixes **`"ab\x00"`** and **`"abc\x00"`**.

In practice, you can assign an integer ID as the prefix. This saves space and lets you rename tables.

0203: Update Modes

We can now map SQL operations to KV. For example, select → get, delete → del.

But the KV set operation seems to be both insert and update. In fact, it is **`insert ... on duplicate update`**. This is a common use case. PostgreSQL adds an **`upsert`** statement to match the KV set operation.

We need to add **`insert`** and **`update`** behavior to KV. Add **`SetEx()`**:

```
type UpdateMode int

const (
    ModeUpsert UpdateMode = 0 // insert or update
    ModeInsert UpdateMode = 1 // insert new
    ModeUpdate UpdateMode = 2 // update existing
)

func (kv *KV) SetEx(key []byte, val []byte, mode UpdateMode) (bool, error)
```

Requirements:

- **`ModeInsert`**: if the key already exists, do not update and return false.
- **`ModeUpdate`**: only update existing keys.
- **`ModeUpsert`**: same as the old **`Set()`**, insert or overwrite:

```
func (kv *KV) Set(key []byte, val []byte) (updated bool, err error) {
    return kv.SetEx(key, val, ModeUpsert)
}
```

0204: CRUD

Add CRUD APIs

This step implements primary-key based CRUD (create, read, update, delete), matching insert, select, update, delete.

```
type DB struct {
    KV KV
}

func (db *DB) Open() error  { return db.KV.Open() }
func (db *DB) Close() error { return db.KV.Close() }
```

Use KV get, set, del to implement:

```
func (db *DB) Select(schema *Schema, row Row) (ok bool, err error)
func (db *DB) Insert(schema *Schema, row Row) (updated bool, err error)
func (db *DB) Upsert(schema *Schema, row Row) (updated bool, err error)
func (db *DB) Update(schema *Schema, row Row) (updated bool, err error)
func (db *DB) Delete(schema *Schema, row Row) (deleted bool, err error)
```

Requirements:

- select, delete: input row contains the primary key.
- insert, upsert, update: input row is complete.

For example, insert:

```
func (db *DB) Insert(schema *Schema, row Row) (updated bool, err error) {
    key := row.EncodeKey(schema)
    val := row.EncodeVal(schema)
    return db.KV.SetEx(key, val, ModeInsert)
}
```

For table **`create table t (a int64, b int64, primary key (b))`**, **`Select()`** takes a **`Row`** of length 2. Column 2 is the primary key. Column 1 is the value returned from KV.

Major Features of Relational Databases

Besides single-row access by primary key, later steps may add:

- Full table scan.
- Query by index.
- Range query by index or primary key.
- Filter query results.

0301: Tokenizer

string → tokens → struct

An SQL string must be parsed into program data before it can be processed. For example:

```
select a,b from t where c=1;
```

Will be represented as:

```
StmtSelect{
    table: "t",
    cols:  []string{"a", "b"},
    keys:  []NamedCell{{column: "c", value: Cell{Type: TypeI64, I64: 1}}},
}
```

SQL is similar to English, with its own words and grammar. In programming languages, words are called **tokens**. Before grammar parsing, the string is split into tokens. This step is called **tokenizing** or **lexing**.

SQL tokens can be grouped as:

- Keywords: select, from, etc.
- Names: table names, column names, etc.
- Symbols: `=`, `,` etc.
- Numbers, strings, etc.

Each type has different rules and is coded in different functions.

Syntax Parser

Most parsing works by consuming tokens from left to right and building data structures. So we need to track the current position in the string.

```
type Parser struct {
    buf string
    pos int
}
func NewParser(s string) Parser {
    return Parser{buf: s, pos: 0}
}
```

Parse Names (table names, column names)

```
func (p *Parser) tryName() (string, bool)
```

Requirements:

- Skip leading spaces.
- First char is a letter or _, following chars are letters, digits, or _.
- On success, return true and advance **`pos`**.
- On failure, return false and keep **`pos`**.

For example, input **`Parser {buf: " hi ", pos: 0}`**. After **`tryName()`**, **`pos = 3`**, return **`"hi"`**.

Use these helpers:

```
func isSpace(ch byte) bool {
    switch ch {
    case '\t', '\n', '\v', '\f', '\r', ' ':
        return true
     }
    return false
}
func isAlpha(ch byte) bool {
    return 'a' <= (ch|32) && (ch|32) <= 'z'
}
func isDigit(ch byte) bool {
    return '0' <= ch && ch <= '9'
}
func isNameStart(ch byte) bool {
    return isAlpha(ch) || ch == '_'
}
func isNameContinue(ch byte) bool {
    return isAlpha(ch) || isDigit(ch) || ch == '_'
}
```

Parse Keywords

```
func (p *Parser) tryKeyword(kw string) bool
```

Requirements:

- Skip leading spaces.
- Match the keyword, case-insensitive. On success, advance **`pos`** and return true.
- Otherwise, return false.
- Keywords must be separated by space or punctuation.

Use this to detect separators:

```
func isSeparator(ch byte) bool {
    return ch < 128 && !isNameContinue(ch)
}
```

0302: Parse Values

Currently only 2 data types are implemented: **int64** and string. They are distinguished by the first charactor:

```
func (p *Parser) parseValue(out *Cell) error {
    p.skipSpaces()
    if p.pos >= len(p.buf) {
        return errors.New("expect value")
    }
    ch := p.buf[p.pos]
    if ch == '"' || ch == '\'' {
        return p.parseString(out)
    } else if isDigit(ch) || ch == '-' || ch == '+' {
        return p.parseInt(out)
    } else {
        return errors.New("expect value")
    }
}
```

When you implement syntax parsing later, you will see that many languages can decide how to parse next based on the first few tokens.

Parse int64

The rules for **int64** are simple: it may start with **-** or **+**, followed by digits.

```
func (p *Parser) parseInt(out *Cell) error
```

Parse Strings

```
func (p *Parser) parseString(out *Cell) error
```

The rules are more complex:

- Enclosed by single or double quotes.
- Quotes inside the string must be escaped with a slash.
- A slash must also be escaped with a slash.

For example, **"is\'nt"** or **'is\'nt'** both mean **isn't**.

To make this practical, more escapes are needed, like **\n**, **\xFF**, **\u3412**. But we have more important topics to learn, so we stop here.

0303: Parse SELECT

Structs

Since only a small set of features has been implemented, our SELECT statement supports only one fixed form: query a single row by primary key. For example, for a table with primary key **`(c, d)`**, the only supported query is this:

```
select a,b from t where c=1 and d='e';
```

Represented as data structures:

```
StmtSelect{
    table: "t",
    cols: []string{"a", "b"},
    keys: []NamedCell{
        {column: "c", value: Cell{Type: TypeI64, I64: 1}},
        {column: "d", value: Cell{Type: TypeStr, Str: []byte("e")}},
    },
}
```

The structs are defined as follows for now and will be extended later:

```
type StmtSelect struct {
    table string
    cols  []string
    keys  []NamedCell
}
type NamedCell struct {
    column string
    value  Cell
}
```

Syntax Parsing

Syntax parsing consumes tokens from left to right while building data structures. For example, parsing **`a = 123`** produces a **`NamedCell`**. It calls **`tryName()`**, **`tryPunctuation()`**, and **`parseValue()`** in order:

```
func (p *Parser) parseEqual(out *NamedCell) error {
    var ok bool
    out.column, ok = p.tryName()
    if !ok {
        return errors.New("expect column")
    }
    if !p.tryPunctuation("=") {
        return errors.New("expect =")
    }
    return p.parseValue(&out.value)
}
```

A new function **tryPunctuation()** is added here. It is similar to **tryKeyword()**, but consumes a punctuation character. The rules are simpler than keywords.

When parsing the **WHERE** part, **parseEqual()** is called in a loop. Even complex syntax becomes easy after splitting it into parts.

Parse SELECT Statements

The leading **select** keyword is used to distinguish SQL statements, though only one is supported now. Then comes a list of column names separated by commas, ending at **from**.

```
func (p *Parser) parseSelect(out *StmtSelect) error {
    if !p.tryKeyword("SELECT") {
        return errors.New("expect keyword")
    }
    for !p.tryKeyword("FROM") {
        if len(out.cols) > 0 && !p.tryPunctuation(",") {
            return errors.New("expect comma")
        }
        if name, ok := p.tryName(); ok {
            out.cols = append(out.cols, name)
        } else {
            return errors.New("expect column")
        }
    }
    if len(out.cols) == 0 {
        return errors.New("expect column list")
    }
    var ok bool
    if out.table, ok = p.tryName(); !ok {
        return errors.New("expect table name")
    }
    return p.parseWhere(&out.keys)
}
```

Parse WHERE

This part only supports **a = 123** joined by **AND**. It is similar to **SELECT a,b**, except commas are replaced by **AND**, and **tryName()** is replaced by **parseEqual()**. You can implement it yourself:

```
func (p *Parser) parseWhere(out *[]NamedCell) error
```

0304: Statements

Other Statements

Following the previous **parseSelect()**, add more statements:

```
create table t (
    a int64,
    b string,
    c string,
    primary key (b, c)
);
insert into t values (1, 'x', 'y');
update t set a = 1 where b = 'x' and c = 'y';
delete from t where b = 'x' and c = 'y';
```

Corresponding types:

```
type StmtCreatTable struct {
    table string
    cols  []Column
    pkey  []string
}
type StmtInsert struct {
    table string
    value [][]Cell
}
type StmtUpdate struct {
    table string
    keys  []NamedCell
    value []NamedCell
}
type StmtDelete struct {
    table string
    keys  []NamedCell
}
```

Only single-row operations by primary key are supported now, so the **WHERE** part maps to **keys**. Range queries and indexes will be added later, with more complex syntax.

Recognize SQL Statements

Add **parseStmt()**, which returns different statement types using **interface{}**:

```
func (p *Parser) parseStmt() (out interface{}, err error) {
    if p.tryKeyword("SELECT") {
        stmt := &StmtSelect{}
        err = p.parseSelect(stmt)
        out = stmt
    } else if p.tryKeyword("CREATE", "TABLE") {
        stmt := &StmtCreatTable{}
        err = p.parseCreateTable(stmt)
        out = stmt
```

```
    } else if p.tryKeyword("INSERT", "INTO") {
        stmt := &StmtInsert{}
        err = p.parseInsert(stmt)
        out = stmt
    } else if p.tryKeyword("UPDATE") {
        stmt := &StmtUpdate{}
        err = p.parseUpdate(stmt)
        out = stmt
    } else if p.tryKeyword("DELETE", "FROM") {
        stmt := &StmtDelete{}
        err = p.parseDelete(stmt)
        out = stmt
    } else {
        err = errors.New("unknown statement")
    }
    if err != nil {
        return nil, err
    }
    return out, nil
}
```

The **tryKeyword()** function is updated to match multiple tokens in sequence:

```
func (p *Parser) tryKeyword(kws ...string) bool
```

Since **parseStmt()** already consumes the leading tokens, the earlier **parseSelect()** must be updated.

This shows that when parsing SQL, looking at the next 1 or 2 tokens is enough to decide what to do. Many languages are this easy to parse and do not need compiler construction theory. Even with syntax like **`select a+b*c`**, recursion is enough.

0305: Execute SQL

Execute Parsed SQL

Some relational database interfaces were implemented earlier:

```
func (db *DB) Select(schema *Schema, row Row) (ok bool, err error)
func (db *DB) Insert(schema *Schema, row Row) (updated bool, err error)
func (db *DB) Update(schema *Schema, row Row) (updated bool, err error)
func (db *DB) Delete(schema *Schema, row Row) (deleted bool, err error)
```

Now SQL is parsed into **`StmtXXX`** types, so these 2 parts can be connected:

```
type SQLResult struct {
    Updated int
    Header  []string
    Values  []Row
}

func (db *DB) ExecStmt(stmt interface{}) (r SQLResult, err error) {
    switch ptr := stmt.(type) {
    case *StmtCreatTable:
        err = db.execCreateTable(ptr)
    case *StmtSelect:
        r.Header = ptr.cols
        r.Values, err = db.execSelect(ptr)
    case *StmtInsert:
        r.Updated, err = db.execInsert(ptr)
    case *StmtUpdate:
        r.Updated, err = db.execUpdate(ptr)
    case *StmtDelete:
        r.Updated, err = db.execDelete(ptr)
    default:
        panic("unreachable")
    }
    return
}
```

`SQLResult` returns the result of a statement:

- SELECT returns **`Header`** and **`Values`**. It is a slice because it will return multiple rows later.
- Other update statements return **`Updated`**, the number of affected rows. For now it can only be 0 or 1.

Store Table Schemas

All operations depend on schemas, so first implement **`execCreateTable()`**. Serialize the table schema and store it under one key. **`DB`** adds a map cache, keyed by table name:

```
type DB struct {
    KV     KV
    tables map[string]Schema
}
```

```
func (db *DB) GetSchema(table string) (Schema, error) {
    schema, ok := db.tables[table]
    if !ok {
        val, ok, err := db.KV.Get([]byte("@schema_" + table))
        if err == nil && ok {
            err = json.Unmarshal(val, &schema)
        }
        if err != nil {
            return Schema{}, err
        }
        if !ok {
            return Schema{}, errors.New("table is not found")
        }
        db.tables[table] = schema
    }
    return schema, nil
}
```

Implement **execCreateTable()**:

1. Convert **StmtCreatTable** to **Schema**.
2. Store **Schema** under the key **@schema_** + table name.
3. Add it to the **DB.tables** map.

```
func (db *DB) execCreateTable(stmt *StmtCreatTable) (err error)
```

Execute SELECT

It is better to split this into small helpers, which can be reused by other statements:

1. **lookupColumns()**: check column names (**select a,b**), return indices in **schema.Cols**.
2. **makePKey()**: check that **WHERE** matches the primary key, return a **Row** with the key filled.
3. **subsetRow()**: return only the columns in **select a,b**.

```
func (db *DB) execSelect(stmt *StmtSelect) ([]Row, error) {
    schema, ok := db.tables[stmt.table]
    if !ok {
        return nil, errors.New("table is not found")
    }
    indices, err := lookupColumns(schema.Cols, stmt.cols)
    if err != nil {
        return nil, err
    }

    row, err := makePKey(&schema, stmt.keys)
    if err != nil {
        return nil, err
    }
    if ok, err = db.Select(&schema, row); err != nil {
        return nil, err
    }
    if !ok {
        return nil, nil
    }
```

```
    row = subsetRow(row, indices)
    return []Row{row}, nil
}
```

Execute INSERT, UPDATE, DELETE

Implement the other SQL statements by calling **`DB.Insert()`**, **`DB.Update()`**, and **`DB.Delete()`**:

```
func (db *DB) execInsert(stmt *StmtInsert) (count int, err error)
func (db *DB) execUpdate(stmt *StmtUpdate) (count int, err error)
func (db *DB) execDelete(stmt *StmtDelete) (count int, err error)
```

Requirements:

- If KV is updated, return **`count = 1`**, otherwise return 0.
- Check that **`StmtXXX`** matches the **`Schema`**.

For UPDATE, we do no support updating the primary keys. UPDATE only changes the V port of KV. Updating a primary key would mean deleting the old key and inserting a new one, which is rarely needed.

0401: Sorting and Search

Sorting and Range Queries

The current SQL can only operate on a single row by primary key. This is limited by the KV interfaces with only get, set, del operations. Now add features:

- Scan a table.
- Query a primary key range, like `a > 123` or `123 < a AND a < 456`.
- Sort results (`ORDER BY`).

These are the same idea: **iterate in sorted order**.

- Table scan: same as range `-∞ < a AND a < +∞`.
- Range query: find bounds, then iterate in order.
- Sorting: `ORDER BY` just controls direction.

Sorting and Search

Sorted data is not only for representation, it also speeds up search. People knew this before the existence of computers. You can quickly find a page in a book because pages are ordered by page numbers. While flipping pages, you run **binary search** unconsciously. The "index" section at the back of a book is also sorted.

Sorting Data Structures

To sort KV, data does not need to be stored in order like book pages. Instead, use **data structures that keep order**. In the current code, disk is an append-only log, memory is a Go map, a hash table. So there is no order. Production KV stores use one of these 3 data structures:

- Hash table: no order, cannot support relational DB.
- Tree, mainly B+Tree: ordered.
- LSM-Tree (Log-Structured Merge Tree): ordered.

The simplest ordered structure is a sorted array. But it is not practical: inserts must move elements, slower as size grows. This is measured by **time complexity**. One insert moves about $N/2$ elements on average, ignore the $1/2$ constant, denoted as $O(N)$. All the structures above, except hash tables, are $O(\log N)$. As N grows, N will always exceed $\log N$.

Arrays are not practical alone, but they evolve into useful data structures. A B+Tree is nested arrays. An LSM-Tree is multiple levels of arrays. Now replace the KV map with a simple array, which will involve into an LSM-Tree later.

```
type KV struct {
    log  Log
    keys [][]byte
    vals [][]byte
}
```

Binary Search

The standard library has binary search, but you should implement it yourself at least once. It looks simple, but many people struggle to code it.

```
func BinarySearchFunc(x S, target T, cmp func(E, T) int) (pos int, ok bool)
```

If the key is found, return ok=true.

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
    if idx, ok := slices.BinarySearchFunc(kv.keys, key, bytes.Compare); ok {
        return kv.vals[idx], true, nil
    }
    return nil, false, nil
}
```

If ok=false, the returned position is where the key should be inserted, which can be used in **SetEx()**.

```
func (kv *KV) SetEx(key []byte, val []byte, mode UpdateMode) (bool, error) {
    idx, exist := slices.BinarySearchFunc(kv.keys, key, bytes.Compare)
    // TODO
}
```

The standard library also provides **slices.Insert()** and **slices.Delete()**.

Database Initialization

Update **KV.Open()** to read the log into arrays:

```
func (kv *KV) Open() error
```

Until LSM-Tree is implemented, all data has to be loaded into memory.

0402: Iterators

Iterator Interface

To traverse an array, you just move an index. With other data structures, traversal is not that simple. Different structures need different methods. So we need a common interface to traverse KV, usually called an iterator:

```
func (kv *KV) Seek(key []byte) (*KVIterator, error)
// Is traversal ended?
func (iter *KVIterator) Valid() bool
// Current element
func (iter *KVIterator) Key() []byte
func (iter *KVIterator) Val() []byte
// Move to next/prev element
func (iter *KVIterator) Next() error
func (iter *KVIterator) Prev() error
```

Usage:

```
for iter, err = kv.Seek(start); err ==nil && iter.Valid(); err = iter.Next() {
    key, val := iter.Key(), iter.Val()
    // ...
}
```

When this becomes disk-based later, IO errors may occur, so they return **`error`**.

Iterator Definition

`KVIterator` needs to track the current position. For an array, this is just an index:

```
type KVIterator struct {
    keys [][]byte
    vals [][]byte
    pos  int // position
}
```

It also keeps references to the slices. If the data structure changes later, this implementation will change completely.

Iterator Implementation

`KV.Seek()` returns an iterator positioned at the result of binary search on **`key`**, that is, the first position where **`key >= target`**.

```
func (kv *KV) Seek(key []byte) (*KVIterator, error) {
    pos, _ := slices.BinarySearchFunc(kv.keys, key, bytes.Compare)
    return &KVIterator{keys: kv.keys, vals: kv.vals, pos: pos},nil
}
```

This position may be out of range, so check with **`Valid()`** before use:

```
func (iter *KVIterator) Valid() bool {
    return 0 <= iter.pos && iter.pos < len(iter.keys)
}
func (iter *KVIterator) Key() []byte { return iter.keys[iter.pos] }
func (iter *KVIterator) Val() []byte { return iter.vals[iter.pos] }
```

Positions **-1** and **len(keys)** mean out of range. With this design, moving one step back when out of range can return to range.

```
func (iter *KVIterator) Next() error {
    if iter.pos < len(iter.keys) {
        iter.pos++
    }
    return nil
}
func (iter *KVIterator) Prev() error {
    if iter.pos >= 0 {
        iter.pos--
    }
    return nil
}
```

Range Queries

KV.Seek() corresponds to the query **a >= key**. For **a > key**, check whether the initial position equals key and move forward by 1. Similarly, **a < key** and **a <= key** can be implemented by adjusting the position from **KV.Seek()**. There is no need to implement all 4 comparison operators separately.

0403: Sort Orders

Primary Key Order

KV keys are compared as strings, using **`bytes.Compare()`**. Relational data has types, and serialized keys may compare in the wrong order.

One fix is to deserialize keys before comparing, then compare by data type. But this makes KV depend on schema, which adds tight coupling. So some databases use another way: **design a serialization format that keeps sort order**.

- Rename the old **`Cell.Encode()`** to **`Cell.EncodeVal()`** for KV values.
- Add **`Cell.EncodeKey()`** for KV keys which implements a new format.
- Update **`Cell.Decode()`** accordingly.

```
// New
func (cell *Cell) EncodeKey(toAppend []byte) []byte
func (cell *Cell) DecodeKey(data []byte) (rest []byte, err error)
// Existing
func (cell *Cell) EncodeVal(toAppend []byte) []byte
func (cell *Cell) DecodeVal(data []byte) (rest []byte, err error)
```

Rework **`Row`** methods to call the new **`Cell`** methods:

```
func (row Row) EncodeKey(schema *Schema) (key []byte)
func (row Row) DecodeKey(schema *Schema, key []byte) (err error)
```

Order-Preserving Serialization

Let's start with a simple problem: How to encode uint64 into 8 bytes so string compare reflects sort order?

String compare starts at byte 0, so it should store the most significant bits (MSB), which is the highest 8 bits. This is **big-endian** encoding. Like sorting dates as yyyy-mm-dd.

But we use int64, not uint64. We can remap int64 to uint64 order, then store as big-endian:

$$(-2^{63}, -2^{63}+1, \ldots, 0, 1, \ldots, 2^{63}-1) \mapsto (0, 1, \ldots, 2^{63}, 2^{63}+1, \ldots, 2^{64}-1)$$

This needs knowledge of signed integer encoding:

- Positive numbers $[0, 2^{63})$ encode the same as uint64.
- Negative numbers $[-2^{63}, 0)$ map to $[2^{63}, 2^{64})$ in uint64.

This swaps the 2 halves of uint64, which is just flipping the MSB. Use **xor** to do it.

```
func (cell *Cell) EncodeKey(out []byte) []byte {
    switch cell.Type {
    case TypeI64:
        return binary.BigEndian.AppendUint64(out,uint64(cell.I64)^(1<<63))
    case TypeStr:
        // TODO
    }
}
```

Floats can be serialized this way too, but note that negative floats differ from integers (see step 0201). The result is simple, think it through yourself.

Null-Terminated Strings

The old length-prefixed string format no longer works. Without length, we must use a **delimiter** to mark the end. For example, add a 0 byte at the end, like C strings.

When comparing 2 null-terminated strings, if one is a prefix of the other, comparison will reach the 0 byte, so the shorter string is smaller, which is correct.

If string data contains 0 bytes, they must be **escaped**. The escape byte itself must also be escaped. Use 0x01 as the escape byte:

- **0x00 ⇔ 0x01 0x01**
- **0x01 ⇔ 0x01 0x02**

```
func encodeStrKey(toAppend []byte, input []byte) []byte {
    for _, ch := range input {
        if ch == 0x00 || ch == 0x01 {
            toAppend = append(toAppend, 0x01, ch+1)
        } else {
            toAppend = append(toAppend, ch)
        }
    }
    return append(toAppend, 0x00)
}
```

This removes 0 bytes from data, keeps order, and is reversible. Implement the decode function yourself:

```
func decodeStrKey(data []byte) (out []byte, rest []byte, err error)
```

Multi-Column Compare

A primary key can have multiple columns. Comparison is like strings, column by column, like **tuple**s in Python. For **`(a, b) > (c, d)`**, it means **`a > c || (a == c && b > d)`**. Question: if you serialize columns and join them, does this still work? The answer is yes.

0404: Row Iterator

Iterator Interface

Now that sort order is right, we can build range queries on top of **`KV.Seek()`**:

```
// Return the first position >= primary key
func (db *DB) Seek(schema *Schema, row Row) (*RowIterator, error)

// Is iteration finished?
func (iter *RowIterator) Valid() bool
// Current row
func (iter *RowIterator) Row() Row
// Move forward
func (iter *RowIterator) Next() error
```

This converts KV pairs from **`KVIterator`** into rows. There is one issue: keys of different tables are split by prefixes, so during scan we must check whether a key belongs to a table. Update **`Row.DecodeKey()`** so that when a key does not belong to the table, it returns **`ErrOutOfRange`**:

```
var ErrOutOfRange = errors.New("out of range")

func (row Row) DecodeKey(schema *Schema, key []byte) (err error) {
    if len(key) < len(schema.Table)+1 {
        return ErrOutOfRange
    }
    if string(key[:len(schema.Table)+1]) != schema.Table+"\x00" {
        return ErrOutOfRange
    }
    // ...
}
```

Iterator Implementation

`RowIterator` is a wrapper around **`KVIterator`**. After moving **`KVIterator`**, we must decode KV and check **`ErrOutOfRange`**, so the decoded row needs to be stored somewhere.

```
type RowIterator struct {
    schema *Schema
    iter   *KVIterator
    valid  bool // decode result (err != ErrOutOfRange)
    row    Row  // decode result
}
```

`Valid()` and **`Row()`** just return stored results:

```
func (iter *RowIterator) Valid() bool { return iter.valid }
func (iter *RowIterator) Row() Row    { return iter.row }
```

Decode the result after the iterator is moved or created:

```
func (iter *RowIterator) Next() (err error) {
    if err = iter.iter.Next(); err !=nil {
        return err
    }
    iter.valid, err = decodeKVIter(iter.schema, iter.iter, iter.row)
    return err
}
```

The remaining functions are left to the reader:

```
func decodeKVIter(schema *Schema, iter *KVIterator, row Row) (bool, error)
func (db *DB) Seek(schema *Schema, row Row) (*RowIterator, error)
```

Summary

Another core database feature is done: range query. Next steps:

- Add SQL support for the new range query feature.
- Implement a real data structure (LSM-Tree).

0405: Range Query

KV Range Query

The **Seek()** function only supports open-ended range queries (**key >= 123**):

```
func (kv *KV) Seek(key []byte) (*KVIterator, error)
```

Now add a close-ended range query API:

```
func (kv *KV) Range(start, stop []byte, desc bool) (*RangedKVIter, error)
```

The **desc** flag controls the traversal order and can be used for **ORDER BY**:

- **desc == false**: query **start <= key && key <= stop**.
- **desc == true**: query **start >= key && key >= stop**.

RangedKVIter just wraps the original **KVIterator**:

```
type RangedKVIter struct {
    iter KVIterator
    stop []byte
    desc bool
}

func (iter *RangedKVIter) Key() []byte {
    return iter.iter.Key()
}

func (iter *RangedKVIter) Val() []byte {
    return iter.iter.Val()
}
```

With a stop condition during iteration:

```
func (iter *RangedKVIter) Valid() bool {
    if !iter.iter.Valid() {
        return false
    }

    r := bytes.Compare(iter.iter.Key(), iter.stop)
    if iter.desc && r < 0 {
        return false
    } else if !iter.desc && r > 0 {
        return false
    }
    return true
}
```

Next() must check the traversal order:

```
func (iter *RangedKVIter) Next() error {
    if !iter.Valid() {
        return nil
    }
    if iter.desc {
        return iter.iter.Prev()
    }else {
        return iter.iter.Next()
    }
}
```

Range() also wraps **Seek()**. Implement it yourself. Note that you may need to adjust the initial iterator position, since **Seek()** is **>=**, while **Range()** can be **>=** or **<=**.

```
func (kv *KV) Range(start, stop []byte, desc bool) (*RangedKVIter, error) {
    iter, err := kv.Seek(start)
    if err !=nil {
        return nil, err
    }
    // TODO
}
```

Prefix Comparison

Now add a similar **Range()** API to **DB**. A new issue appears: a primary key or index can consist of multiple columns, and a query may use only a prefix. For example, if the primary key is (a,b,c), these queries are valid:

- $(a,b,c) \le (123,4,5)$, using the full key. See tuple comparison in step 0403.
- $(a,b) \le (123,4)$, using the (a,b) prefix of (a,b,c), equivalent to $(a,b,c) \le (123,4,+\infty)$.
- $a \le 123$, using the $(a,)$ prefix of (a,b,c), equivalent to $(a,b,c) \le (123,+\infty)$.

If the prefix $(123,4)$ is encoded and used for $K \le (123,4)$ in KV, the actual result is $(a,b,c) < (123,4)$. So we introduce infinity and pad the prefix to a full key. All 4 prefix comparisons can be converted to full key comparisons:

Prefix	Full
$(a,b) < (1,2)$	$(a,b,c) \le (1,2,-\infty)$
$(a,b) \le (1,2)$	$(a,b,c) \le (1,2,+\infty)$
$(a,b) > (1,2)$	$(a,b,c) \ge (1,2,+\infty)$
$(a,b) \ge (1,2)$	$(a,b,c) \ge (1,2,-\infty)$

If we include infinity in KV key encoding, prefix comparison becomes possible. Add a function to encode key prefixes:

```
func EncodeKeyPrefix(schema *Schema, prefix []Cell, positive bool) []byte
```

The **positive** flag decides whether the suffix is $+\infty$ or $-\infty$. Encode infinity as follows:

- Empty string means $-\infty$, the smallest string. This requires serialized data to never be **""**, which is already true.
- **"\xff"** means $+\infty$, the max byte. This requires serialized data to never start with **"\xff"**, which can be enforced by prepending 1 byte to each column.

```
func (row Row) EncodeKey(schema *Schema) []byte {
    key := append([]byte(schema.Table), 0x00)
    for _, idx := range schema.PKey {
        cell := row[idx]
        key = append(key, byte(cell.Type))  // avoid 0xff
        key = cell.EncodeKey(key)
    }
    return append(key, 0x00)                // > -infinity
}
```

Since $-\infty$ is encoded as an empty string, we need to avoid conflicts between (a,b) and $(a,b,-\infty)$ by appending a trailing 0x00 byte, which is greater than $-\infty$.

```
func EncodeKeyPrefix(schema *Schema, prefix []Cell, positive bool) []byte {
    key := append([]byte(schema.Table), 0x00)
    for i, cell := range prefix {
        key = append(key, byte(cell.Type))  // avoid 0xff
        key = cell.EncodeKey(key)
    }
    if positive {
        key = append(key, 0xff)             // +infinity
    }                                       // -infinity
    return key
}
```

DB Range Query

Add a close-ended range query API that supports 4 comparison operators:

```
type ExprOp uint8

const (
    OP_LE ExprOp = 12 // <=
    OP_GE ExprOp = 13 // >=
    OP_LT ExprOp = 14 // <
    OP_GT ExprOp = 15 // >
)

type RangeReq struct {
    StartCmp ExprOp // <= >= < >
    StopCmp  ExprOp
    Start    []Cell
    Stop     []Cell
}

func (db *DB) Range(schema *Schema, req *RangeReq) (*RowIterator, error)
```

The traversal direction is decided by **`StartCmp`**: **`>=`** and **`>`** mean ascending, otherwise descending. **`Start`** and **`Stop`** can be full primary keys or prefixes, and their lengths do not need to match.

Replace the **`KVIterator`** in **`RowIterator`** with **`RangedKVIter`**:

```
type RowIterator struct {
    schema *Schema
    iter   *RangedKVIter // replaced
    valid bool
    row   Row
}
```

The rest is left to the reader:

```
func (db *DB) Range(schema *Schema, req *RangeReq) (*RowIterator, error) {
    start := EncodeKeyPrefix(schema, req.Start, suffixPositive(req.StartCmp))
    stop := EncodeKeyPrefix(schema, req.Stop, suffixPositive(req.StopCmp))
    desc := isDescending(req.StartCmp)
    iter, err := db.KV.Range(start, stop, desc)
    if err !=nil {
        return nil, err
    }
    // TODO
}
```

0501: Infix Expressions

Represent Expressions as Trees

To add range query to SQL, we need to parse more complex syntax. Earlier, we could only parse this fixed form of **WHERE**:

```
select a, b from t where a = 123 and b = 456;
```

Now we need to support these forms:

```
select a, b from t where a > 123;
select a, b from t where (a, b) > (123, 0);
```

We need a data structure to represent the expressions in **WHERE**:

```
type ExprBinOp struct {
    op    ExprOp
    left  interface{}
    right interface{}
}
```

For example:

```
// a = 123 and b = 456
ExprBinOp{op: OP_AND,
    left:  &ExprBinOp{op: OP_EQ,
        left: "a",
        right: &Cell{Type: TypeI64, I64: 123}},
    right: &ExprBinOp{op: OP_EQ,
        left: "b",
        right: &Cell{Type: TypeI64, I64: 456}}}
// a > 123
ExprBinOp{op: OP_GT,
    left:  "a",
    right: &Cell{Type: TypeI64, I64: 123}}
```

Using **interface{}** can refer to any data type. It is similar to **void *** in C/C++, but it keeps type info, so it can be checked and cast back. Here we use 3 kinds of types:

- A string represents a column name.
- **&Cell{}** represents a constant.
- **&ExprBinOp{}** represents a nested expression.

Operator Priority

Why use a tree to represent expressions? Because the tree shows the order of operations. For example, these 2 expressions:

```
a + b - c
a + b * c
```

After parsing into trees, evaluation computes subtrees first, which solves operator priority.

```
((a + b) - c)
(a + (b * c))
```

Infix, Prefix, and Postfix

In `(a + (b * c))`, the operator is in the middle and the operands are on both sides. This form is called an **infix expression**. Because some parentheses can be omitted, it may not look like a tree.

Some languages put the operator first. This is called a **prefix expression**, or S-expression. Prefix expressions map directly to trees. There is no operator priority issue, and all parentheses can even be removed.

```
(+ a (* b c))
+ a * b c
```

Postfix expressions also do not need parentheses, because they directly show the order of operations. They do not even need to be parsed into a tree:

```
(a (b c *) +)
a b c * +
```

Parse Infix Expressions

In this step, we ignore operator priority and only parse expressions with +-:

```
// a
"a"
// a + b
&ExprBinOp{op: OP_ADD, left: "a", right: "b"}
// a + b - 3
&ExprBinOp{op: OP_SUB,
    left:  &ExprBinOp{op: OP_ADD, left:"a", right: "b"},
    right: &Cell{Type: TypeI64, I64:123}}
```

The operands include column names and constants, and return `interface{}`:

```
func (p *Parser) parseAtom() (interface{}, error) {
    if name, ok := p.tryName(); ok {
        return name,nil
    }
    cell := &Cell{}
    if err := p.parseValue(cell); err !=nil {
        return nil, err
    }
    return cell,nil
}
```

Implement the `parseAdd()` function. It loops and calls `parseAtom()` and `tryPunctuation()`:

```
func (p *Parser) parseAdd() (interface{}, error)
```

0502: Expression Evaluation

Interpreter

We can now add computation to SELECT and UPDATE statements:

```
select a + b, c * d from t where a = 123;
update t set x = x + 1 where a = 123;
```

In the previous step, expressions were parsed into a tree structure. During evaluation, we must compute subtrees first, then compute the current node. So this is a recursive function that walks a tree:

```
func evalExpr(schema *Schema, row Row, expr interface{}) (*Cell, error) {
    switch e := expr.(type) {
    case string:
        // TODO
    case *Cell:
        return e, nil
    case *ExprBinOp:
        left, err := evalExpr(schema, row, e.left)
        if err != nil {
            return nil, err
        }
        // TODO
    default:
        panic("unreachable")
    }
}
```

The `evalExpr()` function takes column data (`schema`, `row`), uses a `switch` to check the expression type, and returns the computed result (`*Cell`).

This step is obvious, but it has a name: **tree-walk interpreter**. Many compiler tutorials claim to work in dozens of lines of code. What they implement is simply traversal of trees.

Strong Typing and Weak Typing

When evaluating expressions, types must match. For example, `123 + "abc"` should cause an error. However, some 20th-century designs, such as SQL and JS, auto-convert between types. SQL converts strings to numbers, and uses 0 if conversion fails. This lenient use of types is called **weak typing**. Weak typing was proven to be a bad idea. So some 21st-century databases, such as ClickHouse, do not fully follow SQL and use strict **strong typing**.

You can also check types in advance, instead of checking them in `evalExpr()`. This type-checking function is similar to expression evaluation. It walks the expression recursively, but outputs types instead of values.

0503: Operator Priority

The hard part of infix expressions is operator priority (precedence). There are many methods in Compiler Constructions, but they are not required, because simple recursive calls are enough.

Multiple Priority Levels

This step adds * and /, which have higher precedence than + and -. **Parentheses are ignored for now**. In an expression that includes + - * /, addition and subtraction must be at the upper level (the root) of the tree. That is, subtrees of + and - can contain * and /. But without parentheses, subtrees of * and / cannot contain + or -. They can only be a value or column name, which is what **`parseAtom()`** handled before. So we need 2 changes:

1. Copy **`parseAdd()`** and modify it slightly to create **`parseMul()`**, which parses expressions with only * and /.
2. In **`parseAdd()`**, replace **`parseAtom()`** with **`parseMul()`**, so * and / become subtrees of + and -.

To parse an infix expression with multiple priority levels, start from the lowest priority. The highest priority is at the deepest level of the call chain. Now add **`parseExpr()`** as the entry point for expression parsing:

```
func (p *Parser) parseExpr() (interface{}, error) {
    return p.parseAdd()
}
func (p *Parser) parseAdd() (interface{}, error) {
    // call parseMul()
}
func (p *Parser) parseMul() (interface{}, error) {
    // call parseAtom()
}
```

There are only 2 priority levels here. To add more, repeat the same pattern.

Parentheses

Now add parentheses to change the order of operations. Parentheses can be seen as the highest priority operation, so they should be handled in the deepest call, **`parseAtom()`**. When a parenthesis is found, call the top-level **`parseExpr()`**.

```
func (p *Parser) parseAtom() (expr interface{}, err error) {
    if p.tryPunctuation("(") {
        if expr, err = p.parseExpr(); err !=nil {
            return nil, err
        }
        if !p.tryPunctuation(")") {
            return nil, errors.New("expect )")
        }
        return expr,nil
    }
    // ...
}
```

0504: Expression Parser

SQL Operators

This step adds many SQL operators, with precedence from low to high:

```
a OR b
a AND b
NOT a
a = b, a <> b, a < b, a > b, a <= b, a >= b
a + b, a - b
a * b, a / b
-a
```

Test case:

```
// f or e and not d = a + b * -c
&ExprBinOp{op: OP_OR,
     left: "f", right: &ExprBinOp{op: OP_AND,
         left: "e", right: &ExprUnOp{op: OP_NOT,
             kid: &ExprBinOp{op: OP_EQ,
                 left: "d", right: &ExprBinOp{op: OP_ADD,
                     left: "a", right: &ExprBinOp{op: OP_MUL,
                         left: "b", right: &ExprUnOp{op: OP_NEG,
                             kid: "c"}}}}}}}
```

Extract Common Functions

Like **parseAdd()** in the previous step, each priority level maps to a **parseXXX()** function, which calls the next **parseYYY()**. Earlier, **parseMul()** was copied from **parseAdd()**. Now with many functions, we can extract the shared logic:

```
func (p *Parser) parseOr() (interface{}, error) {
    return p.parseBinop([]string{"OR"}, []ExprOp{OP_OR}, p.parseAnd)
}
func (p *Parser) parseAnd() (interface{}, error) {
    return p.parseBinop([]string{"AND"}, []ExprOp{OP_AND}, p.parseNot)
}
func (p *Parser) parseNot() (expr interface{}, err error)
func (p *Parser) parseCmp() (interface{}, error) {
    return p.parseBinop(
        []string{"=", "!=", "<>", "<=", ">=", "<", ">"},
        []ExprOp{OP_EQ, OP_NE, OP_NE, OP_LE, OP_GE, OP_LT, OP_GT},
        p.parseAdd)
}
func (p *Parser) parseAdd() (interface{}, error) {
    return p.parseBinop([]string{"+", "-"}, []ExprOp{OP_ADD, OP_SUB}, p.parseMul)
}
func (p *Parser) parseMul() (interface{}, error) {
    return p.parseBinop([]string{"*", "/"}, []ExprOp{OP_MUL, OP_DIV}, p.parseNeg)
}
func (p *Parser) parseNeg() (expr interface{}, err error)
```

Extract the differences between **`parseAdd()`** and **`parseMul()`** as parameters:

```
func (p *Parser) parseBinop(
    tokens []string, ops []ExprOp, inner func() (interface{}, error),
) (interface{}, error)
```

Prefix Operators

`NOT a` and **`-a`** have only 1 child, so add a new struct:

```
type ExprUnOp struct {
    op  ExprOp
    kid interface{}
}
```

Note that the child can include operators at the same level, such as **`NOT NOT a`**.

```
func (p *Parser) parseNot() (expr interface{}, err error) {
    if p.tryKeyword("NOT") {
        if expr, err = p.parseNot(); err != nil {
            return nil, err
        }
        return &ExprUnOp{op: OP_NOT, kid: expr}, nil
    } else {
        return p.parseCmp()
    }
}
```

`evalExpr()` should be extended to handle this as well.

0505: SELECT Expression

Modify Structs

In this step, we'll allow expressions in SELECT and UPDATE:

```
select a * 4 - b, d + c from t where d = 123;
update t set a = a - b, b = a, c = d + c where d = 123;
```

In **StmtSelect**, **cols** changes from **string** to **interface{}**. **StmtUpdate** is updated in a similar way, adding **ExprAssign**:

```
type StmtSelect struct {
    table string
    // cols  []string
    cols  []interface{} // ExprUnOp | ExprBinOp | string | *Cell
    keys  []NamedCell
}
type StmtUpdate struct {
    table string
    keys  []NamedCell
    // value []ExprEqual
    value []ExprAssign
}
type ExprAssign struct {
    column string
    expr   interface{} // ExprUnOp | ExprBinOp | string | *Cell
}
```

Modify Parser

- In **parseSelect()**, replace **tryName()** with **parseExpr()**.
- In **parseUpdate()**, replace **parseEqual()** with **parseAssign()**.

```
func (p *Parser) parseAssign(out *ExprAssign) (err error) {
    var ok bool
    out.column, ok = p.tryName()
    if !ok {
        return errors.New("expect column")
    }
    if !p.tryPunctuation("=") {
        return errors.New("expect =")
    }
    out.expr, err = p.parseExpr()
    return err
}
```

Modify Execution

Update select and update to call **evalExpr()** to produce results:

```go
func (db *DB) execSelect(stmt *StmtSelect) ([]Row, error) {
    // ...
    out := make(Row, len(stmt.cols))
    for i, expr := range stmt.cols {
        cell, err := evalExpr(&schema, row, expr)
        if err != nil {
            return nil, err
        }
        out[i] = *cell
    }
    return []Row{out}, nil
}
```

```go
func (db *DB) execUpdate(stmt *StmtUpdate) (count int, err error) {
    // ...
    updates := make([]NamedCell, len(stmt.value))
    for i, assign := range stmt.value {
        cell, err := evalExpr(&schema, row, assign.expr)
        if err != nil {
            return 0, err
        }
        updates[i] = NamedCell{column: assign.column, value: *cell}
    }
    // ...
}
```

0506: WHERE Expression

Parse WHERE

This step adds no new features. It refactor the **WHERE** part into an expression.

```
type StmtSelect struct {
    table string
    cols  []interface{} // ExprUnOp | ExprBinOp | string | *Cell
    // keys  []NamedCell
    cond  interface{}
}

type StmtUpdate struct {
    table string
    // keys  []NamedCell
    cond  interface{}
    value []ExprAssign
}

type StmtDelete struct {
    table string
    // keys  []NamedCell
    cond  interface{}
}
```

Update parser:

```
func (p *Parser) parseSelect(out *StmtSelect) (err error) {
    // ...
    out.cond, err = p.parseWhere()
    return err
}

func (p *Parser) parseWhere() (expr interface{}, err error) {
    if !p.tryKeyword("WHERE") {
        return nil, errors.New("expect keyword")
    }
    if expr, err = p.parseExpr(); err != nil {
        return nil, err
    }
    if !p.tryPunctuation(";") {
        return nil, errors.New("expect ;")
    }
    return expr, nil
}
```

Match Expressions

During execution, extract primary key values from the expression:

```
func (db *DB) execSelect(stmt *StmtSelect) ([]Row, error) {
    // ...
    row, err := matchPKey(&schema, stmt.cond)
    if err !=nil {
        return nil, err
    }
    if ok, err = db.Select(&schema, row); err !=nil {
        return nil, err
    }
    // ...
}
func matchPKey(schema *Schema, cond interface{}) (Row, error) {
    if keys, ok := matchAllEq(cond,nil); ok {
        return makePKey(schema, keys)
    }
    return nil, errors.New("unimplemented WHERE")
}
```

`matchAllEq()` detects expressions like **`a = 123 AND b = 456 AND ...`** and outputs column names and values. It recursively walks the expression. Implement this:

```
func matchAllEq(cond interface{}, out []NamedCell) ([]NamedCell, bool)
```

For now, only primary key queries that return a single row are supported. To add other query types, you must recognize other expression forms, such as range queries in the next step. A **query planner** matches conditions in **`WHERE`** and decides which query to use. If it cannot decide, it scans the full table.

One query can have many equivalent forms. For example, **`(a,b) > (1,2)`** is equivalent to **`a > 1 OR (a = 1 AND b > 2)`**. In practice, databases often fail to recognize some forms, so users must manually rewrite queries. If indexes are added later, expressions must match indexes, and the query planner may still fail. So most OLTP databases allow users to force an index and skip the query planner.

0507: SQL Range Query

Connect to DB.Range()

We can finally add range queries to SQL:

```
select a, b from t where a > 123;
select a, b from t where (a, b) > (123, 0);
```

That is, connect the **WHERE** condition to the existing **DB.Range()**:

```
type RangeReq struct {
    StartCmp ExprOp // <= >= < >
    StopCmp  ExprOp
    Start    []Cell
    Stop     []Cell
}

func (db *DB) Range(schema *Schema, req *RangeReq) (*RowIterator, error)
```

SELECT, UPDATE, and DELETE all need changes. Add **execCond()** to handle **WHERE**:

```
func (db *DB) execCond(schema *Schema, cond interface{}) (*RowIterator, error) {
    req, err := makeRange(schema, cond)
    if err != nil {
        return nil, err
    }
    return db.Range(schema, req)
}

func (db *DB) execSelect(stmt *StmtSelect) (output []Row, err error) {
    // ...
    iter, err := db.execCond(&schema, stmt.cond)
    for ; err == nil && iter.Valid(); err = iter.Next() {
        // ...
    }
    // ...
}
```

The main work of this step is **makeRange()**:

```
func makeRange(schema *Schema, cond interface{}) (*RangeReq, error)
```

Parse Tuples

To support syntax like **(a, b) < (1, 2)**, add a struct and update parenthesis handling:

```
type ExprTuple struct {
    kids []interface{}
}
```

```
func (p *Parser) parseAtom() (expr interface{}, err error) {
    if p.tryPunctuation("(") {
        p.pos--
        return p.parseTuple()
    }
    // ...
}
func (p *Parser) parseTuple() (expr interface{}, err error)
```

Detect Range Queries

Querying **`a = 123`** is the same as **`123 <= a AND a <= 123`**, so single primary key queries can also use range queries. The old **`DB.Select()`** can use this shortcut.

```
func makeRange(schema *Schema, cond interface{}) (*RangeReq, error) {
    if keys, ok := matchAllEq(cond,nil); ok {
        if pkey, ok := extractPKey(schema, keys); ok {
            return &RangeReq{
                StartCmp: OP_GE,
                StopCmp:  OP_LE,
                Start:    pkey,
                Stop:     pkey,
            },nil
        }
    }
    if req, ok := matchRange(schema, cond); ok {
        return req,nil
    }
    return nil, errors.New("unimplemented WHERE")
}
```

Matching expressions is just a set of if-else cases, which the reader can implement:

```
func matchRange(schema *Schema, cond interface{}) (*RangeReq, bool) {
    binop, ok := cond.(*ExprBinOp)
    if ok && binop.op == OP_AND {
        // a > 1 AND a < 2
    }else if ok {
        // a > 1
    }
    return nil, false
}
```

Only a single range is supported now. To support **`OR`**, it would map to multiple **`DB.Range()`** calls, and it is best to simplify range unions and intersections. You get the idea, but the implementation is tendious, so we stop here.

0600: Atomic Updates

From Memory to Disk

Right now, our DB is log + in-memory array. It's an in-memory DB with durability. Data size is limited by memory, so we need disk-based data structures. An array is the simplest structure. We will design a format to save the sorted in-memory array to disk.

But array is not a practical data structure, because inserts and deletes are $O(N)$. There are only 2 practical choices: B+Tree and LSM-Tree. Both can evolve from simple arrays, so we can progress in tiny steps, making the full ideas easier to grasp.

Copy-on-write

Whatever data structure is used, atomicity and durability must be considered once it's on disk, the A and D in ACID. This depends on how updates are done. A log achieves atomicity with checksums because:

- Appending data does not destroy the old state.
- A checksum can detect an incomplete new state.

Array insert and delete move elements and destroy the old array. To avoid destroying the old state, we can copy the data, apply updates (still $O(N)$), then replace the old array with the new one. The **replace** step can be atomic. For example, Linux `rename()` can replace a file atomically. This copy + replace method is called **copy-on-write** (CoW).

On Linux, after a crash, the target of `rename()` is either the old file or the new file. Provided that the data is `fsync`ed before renaming. Also `fsync` the directory (see step 0104).

This atomicity applies only to the target file, not deletion of the source file. There is an intermediate state where both file names point to the same data. This is fine as long as the source file name is not reused.

Root Pointer

For tree structures like B+Tree, CoW does not need to copy all data. Only the path from leaf to root is copied, other subtrees are shared, then the root pointer is replaced (atomically). We won't pursue the details here.

If full updates are done by replacing files, the file name that points to the data can be considered a "root pointer". **Atomic data update becomes atomic pointer update**.

```
0. Initial state            pointer -> [ old data ]

1. Write new version        pointer -> [ old data ]
                                       [ new data ]
2. fsync new version
3. Switch to new (atomic)              [ old data ]
                             pointer -> [ new data ]

4. Delete old version       pointer -> [ new data ]
```

The common idea of copy-on-write and logs is: **write new data without destroying old data, then switch atomically**.

a. For append-only logs, switching is done implicitly via checksums.
b. For files, atomic file rename can be used.
c. For trees, switching the root pointer can use a single-sector atomic write.

Double Buffering

There is another method that does not rely on filesystem or hardware atomicity:

- Store 2 copies of data, update them in turn.
- Each copy has an monotonic version number, the copy with greater version is used.
- Each copy has a checksum, so the potential bad copy is ignored after recovery.

```
slot 0 -> [ version 124 | data yyy | crc32 ]
slot 1 -> [ version 123 | data zzz | crc32 ]
```

This is a cyclic log with only 2 records (ring buffer). It's ideal for a root pointer.

Double Write

The methods above simply avoid **inplace update**, but they either do full updates or need a root pointer. For data structures without a root pointer, like hash tables, there are other ways to do incremental updates.

A hash table is an array that needs inplace update, which destroys old data. If we save the old data before overwriting it, we can always restore the previous state after a crash. Example: at some offset, write n bytes yyy, overwrite xxx:

1. Save a record: at this offset, n bytes xxx will be overwritten, and a checksum.
2. `fsync` the record.
3. Overwrite xxx with yyy.
4. `fsync` the data.

- If a crash happens in steps 3 or 4, the data structure is in an unknown state, just use the record to revert to xxx at that offset.
- If a crash happens in steps 1 or 2, the checksum ensures the atomicity of the record. If the record is bad, then the data structure must be good because the data was not written. If the record is good, then use it to fix the (potentially bad) data structure.

Instead of saving old data xxx, the record can also save new data yyy. Then recovery restores the new state yyy instead of the old state xxx. Both methods write data twice, so this is called **double write**.

These 2 methods are like logs, but they record low-level writes, not logical operations. So recovery works even when the data structure is in an unknown state. This is also called **physical logging**.

If restoring to new state, the DB can return to the client after the first `fsync` and complete the rest steps asynchronously.

Log-structured Merge Tree

All above methods are about updating data. There is a way to achieve updates without updating data: the log-structured merge tree (LSM-Tree). It's often compared with B+Tree, but it's not a concrete data structure. It's a set of structures with 3 operations:

- On update, add a new data structure to the set.
- **Merge** existing data structures to keep the set size from unlimited growth.
- On query, search all data structures and **merge** results.

Action	Set	Set size
Initial	empty	0
set a=1	[a=1]	1
set a=2	[a=2] [a=1]	2
merge	[a=2]	1
set c=3	[c=3] [a=2]	2
merge	[c=3, a=2]	1
set d=4	[d=4] [c=3, a=2]	2
set e=5	[e=5] [d=4] [c=3, a=2]	3
merge	[e=5, d=4] [c=3, a=2]	2
merge	[e=5, d=4, c=3, a=2]	1
set f=6	[f=6] [e=5, d=4, c=3, a=2]	2
set a=0	[a=0] [f=6] [e=5, d=4, c=3, a=2]	3
merge	[a=0, f=6] [e=5, d=4, c=3, a=2]	2

From this example, you might have guessed that merging data structures of similar size makes element size grow exponentially, and the set size stays at $O(\log N)$. Multiple versions of a key may exists in different "levels", and queries use the newest one. Details come later.

Any data structure that supports **merge** can be included: arrays, hash tables, trees. The name LSM-Tree is misleading, since it has nothing to be with log or tree. An LSM with hash tables is sometimes called a log-structured hash table, which is a even worse name since it drops the key info "merge".

LSM never modifies data structures. It only adds or removes data structures from the set. The set is just pointers to data structures, which can be updated atomically with the methods above.

What Is the Roles of the Log?

Adding disk-based data structures solves the problem of unbounded log growth. When the log reaches a limit, data is moved to disk data structures and the log is reset. The log itself cannot be queried, so it's mirrored in an in-memory data structure, often called a MemTable. When the log is reset, the MemTable is reset too, keeping memory bounded.

The log must eventually move data to disk structures. Why not update disk structures directly with the methods above? Because logs improve performance: each append needs only 1 `fsync`, while the other methods need at least 2 `fsync`s unless hardware atomics are used. Batching multiple updates with a log reduces the average `fsync` count to nearly 1.

Comparison of Atomic Update Methods

- Log + checksum: just a log, not general. Needs 1 `fsync`.
- Copy-on-write: full update for small data, or incremental update for trees. Needs another way to switch the root pointer atomically. At least 2 `fsync`.
- Double buffering: full update for small data. Needs 1 `fsync`.
- Double write: incremental update for any data structure. Needs 2 `fsync`.
- LSM-Tree: for mergeable data structures, avoids updates. Not a concrete solution.

0601: Build SSTables

SSTable Format

In this step, we persist the sorted in-memory array to a file. This file is initially created by flushing the in-memory data structure (mirroring the log). Later, whenever the log reaches a size limit, it is merged with the old file into a new file, then replace the old one.

Because of the log, we do not need to rewrite the entire file on every update. However, amortized complexity is still $O(N)$. This cannot be solved by a single file, but the file format can be reused later in the LSM-Tree.

In an LSM-Tree, the on-disk data structure is usually called an SSTable (sorted string table), because it supports seek and iteration in sort order. It's not just a bunch of sorted strings, it can contain a concrete data structure, like an array or a tree. An SSTable is just some implementation details; even with the same data structure, different on-disk formats can be used. We adopt the following format:

```
[ n keys | offset1 | offset2 | ... | offsetn | KV1 | KV2 | ... | KVn ]
  8bytes   8bytes
```

First comes an array that records the offset of each key in the file. The array size (n) is stored at the beginning. This array is used for binary search and iteration. Each KV is stored in the following format:

```
[ key length | val length | key data | val data ]
    4bytes       4bytes
```

This format is a bit redundant, since the size of K+V can be inferred from adjacent offsets.

Constructing an SSTable

The initial SSTable is created by flushing the in-memory data structure (MemTable). When building an SSTable, input data is fed through an iterator, so the code does not depend on the specific data type.

```go
type SortedFile struct {
    FileName string
    fp       *os.File
}

func (file *SortedFile) CreateFromSorted(kv SortedKV) error

type SortedKV interface {
    Size() int
    Iter() (SortedKVIter, error)
}
type SortedKVIter interface {
    Valid() bool
    Key() []byte
    Val() []byte
    Next() error
    Prev() error
}
```

This iterator is an interface and matches the existing **KVIterator**. Using an interface also makes it more testable; see the test cases for details.

The input interface also provides the number of keys. With this, we know the size of the offset array and therefore the offset of the first KV. We can write both the offset array and the KV data in a single pass. To write at a specific position, use **WriteAt()**:

```
func (fp *os.File) WriteAt(b []byte, offset int64) (n int, err error)
```

The OS tracks a "current position" for an open file. Calling **Write()** automatically advances it. The position can be changed with **Seek()**, but **WriteAt()** is just more convenient.

IO Buffering

On Linux, writes first goes into the page cache. The OS decides when to flush it to disk, unless **fsync** is used. If the same page is modified repeatedly, it does not trigger disk writes every time, meaning writes might be merged.

When writing an SSTable, each KV causes multiple writes, and 2 of them are only 8 bytes. This likely modifies the same pages repeatedly, which benefits from the page cache. However, each syscall has overhead, so for small consecutive writes, it is better to buffer at the application level as well.

In the C standard library, **fwrite()** is buffered by default. In Go, file **Write()** maps to OS syscall. To get buffering, wrap the file with **bufio.NewWriter()**.

```
w := bufio.NewWriter(fp) // bufio.Writer{}
w.Write(data)
// ...
w.Flush()   // write buffered data (if any)
fp.Sync()   // fsync
```

The problem is that **bufio.Writer** assumes sequential writes, while we are using **WriteAt()**. If we construct the SSTable in one pass, we are still doing sequential writes in two regions (the offset array and the KV data), so we can use two **bufio.Writer**s. This does not require opening the file twice; instead, we can wrap **WriterAt()** as an **io.Writer** interface.

```
type OffsetWriter {
    writer io.WriterAt
    offset int64
}
func (w *OffsetWriter) Write(data []byte) (n int, err error) {
    n, err = w.writer.WriteAt(data, w.offset)
    w.offset += int64(n)
    return n, err
}
```

The **io.WriterAt** interface is provided by the standard library:

```
type WriterAt interface {
    WriteAt(p []byte, off int64) (n int, err error)
}
```

0602: Query SSTable

Reading an SSTable

The previous step effectively serialized `[]struct{ key []byte; val []byte }` to disk. This format can be used directly without deserialization and does not need to be fully loaded into memory. Now we implement reading the n-th KV pair from the array:

```
type SortedFile struct {
    FileName string
    fp       *os.File
    nkeys    int // added
}

func (file *SortedFile) index(pos int) (key []byte, val []byte, err error) {
    var buf [8]byte
    if _, err = file.fp.ReadAt(buf[:], int64(8+8*pos)); err != nil {
        return nil, nil, err
    }
    // KV offset
    offset := int64(binary.LittleEndian.Uint64(buf[:]))
    if int64(8+8*file.nkeys) > offset {
        return nil, nil, errors.New("corrupted file")
    }
    // read KV
    if _, err = file.fp.ReadAt(buf[:], offset); err !=nil {
        return nil, nil, err
    }
    // TODO
}
```

`ReadAt()` reads data from a specific file offset, the sibling of **`WriteAt()`**.

Seek and iterate

Using **`index()`**, we then implement binary search and iterators. Because data is on disk, IO errors may occur, so **`error`** is returned.

```
func (file *SortedFile) Seek(key []byte) (SortedKVIter, error) {
    // ...
    iter := &SortedFileIter{file: file, pos: pos}
    // ...
    return iter,nil
}
```

From now on, prefer returning interfaces instead of concrete types, which avoids tight decoupling. The iterator implementation is similar to the existing **`KVIterator`**:

```
type SortedFileIter struct {
    file *SortedFile
    pos  int
    key  []byte
    val  []byte
}
```

```
func (iter *SortedFileIter) Valid() bool {
    return 0 <= iter.pos && iter.pos < iter.file.nkeys
}
func (iter *SortedFileIter) Key() []byte { return iter.key }
func (iter *SortedFileIter) Val() []byte { return iter.val }
func (iter *SortedFileIter) Next() error
func (iter *SortedFileIter) Prev() error
```

Errors that occur while reading KVs are returned via **`Prev()`** and **`Next()`**. So after moving **`pos`**, the KV must be read and stored in the iterator.

mmap

There is **`bufio.Reader{}`** for cached sequential reads, corresponding to **`bufio.Writer{}`**, which can reduce syscalls. However, binary search requires random access. To cache random IO in application, data must be managered like the page cache, with an algorithm for evicting pages to make room for new pages. This is much more complex than caching sequential IO. So no standard solution for caching random IO. Production DBs may implement their own IO cache and use direct IO to bypass the page cache.

Some databases do not implement an IO cache and instead use **`mmap`** to leverage the page cache, such as LMDB. **`mmap`** is an OS feature that maps a file to a memory address and returns a **`[]byte`**. Reading or writing this memory is equivalent to reading or writing the file. It can be done with **`syscall.Mmap()`**:

```
func Mmap(fd int, offset int64, len int, prot int, flags int) ([]byte, error)
```

`mmap` essentially exposes the page cache to the app, skipping read/write calls. The CPU can tell the OS which pages the app has accessed. Linux then knows which pages were modified and need to be flushed to disk. When the app accesses a page that is not present, a read is automatically triggered to fill it.

Using **`mmap`** not only reduces syscall overhead but also simplifies code. The cost is that errors handling is gone, since there are no read/write calls that can return errors. When an IO error occurs, Linux sends a signal to kill the process, which might not be acceptable.

0603: Refactor Code

If SSTables are added to KV, queries must read both the in-memory structure (MemTable) and SSTables, then merge results. The current in-memory structure is just 2 arrays:

```
type KV struct {
    log  Log
    keys [][]byte
    vals [][]byte
}
```

This step adds no new features. It moves the in-memory structure into a separate type to prepare for the next step. It also makes it easier to replace the data structure later.

```
type KV struct {
    log Log
    mem SortedArray
}
type SortedArray struct {
    keys [][]byte
    vals [][]byte
}
```

The extracted **SortedArray** implements the **SortedKV** interface and can be used as input for SSTables:

```
func (arr *SortedArray) Size() int { return len(arr.keys) }
func (arr *SortedArray) Iter() (SortedKVIter, error) {
    return &SortedArrayIter{arr.keys, arr.vals,0}, nil
}
```

Rename **KVIterator** to a more fitting name: **SortedArrayIter**. Change **KV.Seek()** to return an interface:

```
func (kv *KV) Seek(key []byte) (SortedKVIter, error)
```

Do not access struct fields directly. Use methods instead. Move the original get, set, del there as well:

```
func (arr *SortedArray) Key(i int) []byte { return arr.keys[i] }
func (arr *SortedArray) Clear()
func (arr *SortedArray) Push(key []byte, val []byte)
func (arr *SortedArray) Pop()
func (arr *SortedArray) Get(key []byte) (val []byte, ok bool, err error)
func (arr *SortedArray) Set(key []byte, val []byte) (updated bool, err error)
func (arr *SortedArray) Del(key []byte) (deleted bool, err error)
```

0604: Merge Sorted Data

Merge KV

If SSTables were added, there will be 2 data structures. Keys first go into the MemTable and are later merged into SSTables, so queries may need to merge results from both.

A key may exist in both the MemTable and an SSTable, the MemTable has higher priority. When SSTables are merged later, duplicate keys are dropped, otherwise the DB will grow unlimited like a log.

Since a key may exist in both MemTable and SSTable, we cannot directly delete a key from the MemTable, or the lower-level SSTable would become visible. The MemTable must record deleted keys. This is considered in the next step and is ignored for now.

In the LSM-Tree to be implemented later, there can be multiple levels of SSTables. Lower-level SSTables are merged from upper levels. When querying a single key, search from top to bottom, level by level. This is why the first popular LSM-Tree DB was called LevelDB.

Merge Ranges

Single-key queries can check each level in order. Range queries cannot, because the next key may come from any level. To iterate in ascending order, you must pick the smallest key among all level iterators. This is exactly merge sort.

When talking about sorting, you may think of array sorting algorithms like quick sort or bubble sort. These are **array sorting** methods. Merge sort does not require arrays; it only requires the data to be iteratable. In LSM-Trees, range queries and SSTable merging both are merge sort.

Merge Sort

We implement merging of k ranges, which can be reused later for the LSM-Tree. It is not more complex than merging 2 ranges.

```
type MergedSortedKV []SortedKV

func (m MergedSortedKV) Iter() (iter SortedKVIter, err error) {
    levels := make([]SortedKVIter, len(m))
    for i, sub := range m {
        if levels[i], err = sub.Iter(); err != nil {
            return nil, err
        }
    }
    return &MergedSortedKVIter{levels, levelsLowest(levels)},nil
}

type MergedSortedKVIter struct {
    levels []SortedKVIter
    which int
}
func (iter *MergedSortedKVIter) Valid() bool {
    return iter.which >= 0
}
```

```
func (iter *MergedSortedKVIter) Key() []byte {
    return iter.levels[iter.which].Key()
}
func (iter *MergedSortedKVIter) Val() []byte {
    return iter.levels[iter.which].Val()
}
func (iter *MergedSortedKVIter) Next() error
func (iter *MergedSortedKVIter) Prev() error
```

which records the iterator with the current smallest or largest key. Requirements:

- Upper levels have higher priority than lower levels.
- Do not output duplicate keys.
- Direction can change midway.

Bloom Filter

A single-key query is a special case of a range query. So **KV.Get()** can simply call **KV.Seek()**. But in LSM-Trees, single-key queries can be optimized further.

When analyzing algorithm complexity, we consider the randomized average case. For random key lookups, a key is likely in the bottom level, so the time complexity is $O(\log N)$. In real workloads, access is often not uniformly random. Some workloads mostly query recently updated data, and LSM-Tree levels are ordered by recency. In this case, keys are likely in the top level. Searching from top to bottom gives the best case time complexity $O(1)$, while **KV.Seek()** requires all levels in all cases.

Single-key queries can be further optimized with Bloom filters, which is a compact data structure that can output 2 results in $O(1)$ time:

a. A key definitely does not exist.
b. A key probably exists.

It can be seen as a lossy compressed hash table. Details are left to the reader. If each SSTable has a Bloom filter, most levels where the key does not exist can be skipped. A Bloom filter has these properties:

- Space usage is $O(N)$. Compared to SSTables, it is still small and likely cached in memory.
- Time complexity is $O(1)$. This does not affect overall complexity, though.
- Keys can only be added, not deleted. This is not an issue because SSTables are immutable.

0605: Log + SSTable

We'll add SSTables in this step. Queries must merge results from the MemTable and the SSTable. We introduce **`KV.Compact()`**, which merges the MemTable (mirroring the log) and the SSTable to produce a new SSTable.

```
type KV struct {
    log  Log
    mem  SortedArray
    main SortedFile // added
}

func (file *SortedFile) Open() error
func (kv *KV) Compact() error
```

Deleted Flag

Now that there are 2 levels of data structures, deleted keys must be recorded in the upper level; otherwise, queries would return keys from the lower level.

```
type SortedArray struct {
    keys    [][]byte
    vals    [][]byte
    deleted []bool  // added
}

type SortedArrayIter struct {
    keys    [][]byte
    vals    [][]byte
    deleted []bool  // added
    pos     int
}

func (iter *SortedArrayIter) Deleted() bool { return iter.deleted[iter.pos] }
```

This flag currently exists only in the top-level MemTable; the bottom-level SSTable does not need it. Later, when implementing an LSM-Tree with multiple SSTable levels, intermediate levels will also need this flag.

Related functions in **`SortedArray`** need to be modified, including del and set.

```
func (arr *SortedArray) Clear()
func (arr *SortedArray) Push(key []byte, val []byte, deleted bool)
func (arr *SortedArray) Pop()
func (arr *SortedArray) Del(key []byte) (deleted bool, err error)
func (arr *SortedArray) Set(key []byte, val []byte) (updated bool, err error)
```

Merge Range Queries

Following **`MergedSortedKV.Scan()`**, add **`MergedSortedKV.Seek()`**, which initializes the iterator with each level's **`Seek()`**:

```
func (m MergedSortedKV) Seek(key []byte) (iter SortedKVIter, err error) {
    levels := make([]SortedKVIter, len(m))
    for i, sub := range m {
        if levels[i], err = sub.Seek(key); err != nil {
            return nil, err
        }
    }
    return &MergedSortedKVIter{levels, levelsLowest(levels)},nil
}
```

The merged range query:

```
func (kv *KV) Seek(key []byte) (SortedKVIter, error) {
    m := MergedSortedKV{&kv.mem, &kv.main}
    iter, err := m.Seek(key)
    if err != nil {
        return nil, err
    }
    return filterDeleted(iter)
}
```

Output must filter out deleted keys. We can wrap the iterator to do this:

```
func filterDeleted(iter SortedKVIter) (SortedKVIter, error) {
    for iter.Valid() && iter.Deleted() {
        if err := iter.Next(); err != nil {
            return nil, err
        }
    }
    return NoDeletedIter{iter},nil
}

type NoDeletedIter struct {
    SortedKVIter   // inherits all methods
}

// override Next() and Prev()
func (iter NoDeletedIter) Next() (err error) {
    err = iter.SortedKVIter.Next()
    for err == nil && iter.Valid() && iter.Deleted() {
        err = iter.SortedKVIter.Next()
    }
    return err
}
```

Modify the SSTable Format

When building an SSTable in one pass, we need know the key count to allocate the offset array. However, after merging, duplicate keys and deleted keys are eliminated, so the final key count is smaller than the sum of key counts in all levels. Thus, the offset array size is only an estimate, but it still works as long as we write the true key count after iteration. We need to slightly modify **`SortedFile.CreateFromSorted()`**. Also, **`SortedKV.Size()`** is renamed to a more fitting name.

```
type SortedKV interface {
    EstimatedSize() int                       // renamed
    Iter() (SortedKVIter, error)
    Seek(key []byte) (SortedKVIter, error)  // added
}

func (m MergedSortedKV) EstimatedSize() (total int) {
    for _, sub := range m {
        total += sub.EstimatedSize()
    }
    return total
}
```

Over-allocating the offset array creates an unused gap in the file:

```
[ n keys | offset1 | offset2 | ... | offsetn | gap | KV1 | KV2 | ... | KVn ]
```

You might think this gap wastes disk space, but it's not true. Filesystems do not always allocate space for the entire file. For example, writing 1 MB at offset 0 and 2 MB at offset 5 MB yields a 7 MB file that actually consumes 3 MB of disk space. Reading from the 4 MB gap returns zeros. Most modern filesystems support this feature, called a **sparse file**. The gap is called a **hole**.

Move the Log to an SSTable

`KV.Compact()` merges the MemTable into the SSTable in 3 steps:

```
func (kv *KV) Compact() error {
    // 1. Merge MemTable and SSTable, output to a temporary file.
    path := randomTempPath()
    defer os.Remove(path)
    file := SortedFile{FileName: path}
    m := MergedSortedKV{&kv.mem, &kv.main}
    if err := file.CreateFromSorted(m); err !=nil {
        return err
    }
    // 2. Replace the original SSTable (atomic operation).
    if err := renameSync(file.FileName, kv.main.FileName); err !=nil {
        _ = file.Close()
        return err
    }
    file.FileName = kv.main.FileName
    _ = kv.main.Close()
    kv.main = file
    // 3. Drop the MemTable and the log.
    kv.mem.Clear()
    return kv.log.Truncate()
}
```

rename()

In case of a power failure, Linux's **`rename()`** syscall guarantees atomicity for the target file. However, to make it durable, you must also **`fsync`** the parent directory (see step 0104).

```
func syncDir(file string) error {
    flags := os.O_RDONLY | syscall.O_DIRECTORY
    dirfd, err := syscall.Open(path.Dir(file), flags, 0o644)
    if err != nil {
        return err
    }
    defer syscall.Close(dirfd)
    return syscall.Fsync(dirfd)
}

func renameSync(src string, dst string) error {
    if err := os.Rename(src, dst); err != nil {
        return err
    }
    return syncDir(dst)
}
```

On Windows, this is more complicated: the atomic replacement API is different, and you cannot replace an open file. On embedded systems, atomic operations may not even be available. A more general approach is for the DB to record the new file name itself, rather than relying on a fixed file name. How to record this file name is discussed in section 0600.

ftruncate()

Resetting a log file means seeking to the beginning and **`Truncate()`** the file to 0 size to release disk space. This corresponds to Linux's **`ftruncate()`** syscall.

```
func (log *Log) Truncate() error {
    if _, err := log.fp.Seek(0, io.SeekStart); err != nil {
        return err
    }
    return log.fp.Truncate(0)
}
```

`Compact()` only shows how the log integrates with the data structure; the data structure itself is not yet practical. A real **`Compact()`** will be implemented later with the LSM-Tree.

0700: LSM-Tree Introduction

What LSM-Tree Is and Is Not

Log-structured Merge Tree and B+Tree are the only 2 choices to build a general-purpose DB. LSM-Tree can be compared with B+Tree, but it is not a concrete data structure. It is a method that replaces "updating" data structures with "merging". The name is misleading, because its idea has nothing to do with trees or logs.

Our database have 2 levels: MemTable and SSTable. MemTable uses a log for durability, while SSTable is never modified and is only replaced by new files. For queries, results are merged, and the upper MemTable has higher priority. For updates, data goes to the upper level first, then to lower levels.

This 2-level structure can be extended to k levels, which is the LSM-Tree. What each level contains is irrelevant to the LSM-Tree principle. Each level only needs to be a set of sorted KV, which can be an array or a tree. If ordering is not required, it can even be a hashtable. So LSM-Tree is a collection of data structures, not a single one.

```
level 1     [x=1, z=3]
level 2     [a=8, y=2, z=(deleted)]
...
level k     [a=0, b=2, c=3, d=4, z=456]
```

k-Way Merge Sort

A key may exist in multiple levels. Upper levels are newer, so queries go from top to bottom. Deleted keys must be recorded to prevent exposing old versions from lower levels. Range queries on KV are just k-way merge sort (see step 0604).

Merge Instead of Update

Suppose we have an array of size N. Insert or delete requires moving $N/2$ elements on average. That is $O(N)$, the same as rebuilding the array. With full rebuilds, we can atomically switch to the new array using methods from 0600. The problem is the high time cost.

Databases use logs to reduce update cost. If we rebuild after accumulating n entries, the average update cost becomes $1/n$ of the original. But the complexity is still linear. To reduce time complexity, as N grows, merge frequency must decrease.

A similar case is appending to a dynamic array. If capacity is exceeded, space is reallocated and data is moved. If capacity grows exponentially, from 0 to N, the moved elements are $4,8,16,\ldots$, which sums to $O(N^2)$. The average append cost is $O(N)$.

If merges are triggered exponentially, for example when the log reaches N entries and merges with an array of same size N, the average cost becomes linear. The next issue is that the log is mirrored in memory. Its size cannot grow linearly, or the system becomes an in-memory DB. This is why LSM-Tree needs multiple levels.

Set the log size limit to a constant n. When reached, move to a new array of size n as level 2 (log is level 1), reset the log. When another size n array appears, the 2 adjacent levels are merged into a size $2n$ array. Continue inserting keys until another size $2n$ array appears, then merge into $4n$, and so on.

An example of LSM-Tree growth (level 1 limit is set to 1):

Operation	levels	level count	level sizes
set a=2	[a=2]	1	1
set c=3	[c=3] [a=2]	2	1 1
merge	[c=3, a=2]	1	2
set d=4	[d=4] [c=3, a=2]	2	1 2
set e=5	[e=5] [d=4] [c=3, a=2]	3	1 1 2
merge	[e=5, d=4] [c=3, a=2]	2	2 2
merge	[e=5, d=4, c=3, a=2]	1	4
set f=6	[f=6] [e=5, d=4, c=3, a=2]	2	1 4
set a=0	[a=0] [f=6] [e=5, d=4, c=3, a=2]	3	1 1 4
merge	[a=0, f=6] [e=5, d=4, c=3, a=2]	2	2 4

Level sizes grow exponentially, so the number of levels is at most $\log N$. Data in the bottom comes from the top and is written at most $\log N$ times. So update is $O(\log N)$ on average.

Operation Complexity

The example above uses array + log, but the idea does not restrict data structures. The only requirement is that merge cost is linear. The data structures does not even need to implement updates. This is why we can start from simple arrays.

If each level is key-sorted, lookup cost per level is $O(\log N)$. Across all levels it is the sum of $\log 1 + \log 2 + \log 4 ... + \log N$, which is still $O(\log N)$.

If hashtables are used, average update and lookup cost is $O(\log N)$, not $O(1)$. This is because it is a collection of hashtables, not a single one.

Concrete Data Structures

Levels are usually stored as files. To avoid creating new files on every update, level 1 is often directly updatable. The simplest form is a log. The log must be mirrored in an in-memory structure (MemTable) for queries. That is why almost all LSM-Tree docs mentions these components.

Deleted keys take space until reaching the last level, which makes the last level special.

Arrays use binary search, but real systems should use n-ary search to reduce IO. Operating systems read disks in pages. To use information efficiently, each n-ary branch should use at least one full page. This is essentially a static B-Tree. Readers can explore details themselves. B+Tree is the most practical B-Tree variant, and in databases B-Tree actually means B+Tree. Since LSM-Tree is a set of data structures, it can contain B+Tree as well.

LSM-Tree is much easier to implement than B+Tree. A simple array is enough. Even if it contains B+Trees, a static, non-updatable B+Tree is far simpler than a full B+Tree.

0701: Atomic Store

Double Buffering

The previous step had only 1 SSTable level, with a fixed file name, replaced via **`rename()`**. An LSM-Tree has multiple SSTable levels and multiple files, so fixed names do not work. The list of file names of levels must be recorded by the DB.

Since the number of levels does not exceed $\log N$, the list of file names does not need incremental updates. We can use the double buffering from 0600 to achieve atomic updates. That is, keep 2 copies with version and checksum, and update them in turn. After power loss, we can recover to the good version.

```
slot 0 -> [ version 124 | data yyy | crc32 ]
slot 1 -> [ version 123 | data zzz | crc32 ]
```

Metadata

These 2 records can be 2 files. Define the structs as follows:

```go
type KVMetaStore struct {
    slots [2]KVMetaItem
}
type KVMetaItem struct {
    FileName string
    fp       *os.File
    data     KVMetaData
}
type KVMetaData struct {
    Version uint64
    SSTable string  // file name
}
func (meta *KVMetaStore) Open() error
func (meta *KVMetaStore) Close() error
func (meta *KVMetaStore) Get() KVMetaData
func (meta *KVMetaStore) Set(data KVMetaData) error
```

`KVMetaData` is the data to store, which has an monotonic version and an SSTable file name. Write it to the file using any serialization method (JSON is fine). Requirements:

- **`Open()`** ignores bad copy, including deserialize errors and checksum mismatch.
- **`Get()`** returns the copy with the larger version.
- **`Set()`** overwrites the copy with the smaller version and calls **`fsync`**.

0702: Store Metadata

Directory Layout

Since the DB uses multiple files, it should manage them itself. The user only specifies a directory:

```
type KVOptions struct {
    Dirpath string
}

type KV struct {
    Options KVOptions
    meta    KVMetaStore
    log     Log
    main    SortedFile
    // ...
}
```

SSTable file names are generated dynamically and stored in **KVMetaData**. While **Log** and **KVMetaData** use fixed file names:

```
    kv.log.FileName = path.Join(kv.Options.Dirpath, "kv_log")
    kv.meta.slots[0].FileName = path.Join(kv.Options.Dirpath, "meta0")
    kv.meta.slots[1].FileName = path.Join(kv.Options.Dirpath, "meta1")
```

Record SSTable File Names

KV.Compact() used to replace files via **rename()**. Now it uses **KVMetaStore** to record the new file name, and deletes the old file after success.

```
type KV struct {
    version uint64
    // ...
}

func (kv *KV) Compact() error {
    kv.version++
    sstable := fmt.Sprintf("sstable_%d", kv.version)
    file := SortedFile{FileName: path.Join(kv.Options.Dirpath, sstable)}
    // ...
}
```

The file name is generated by an monotonic number, which is the version in **KVMetaData**.

```
func (kv *KV) Open() error {
    // ...
    kv.version = kv.meta.Get().Version
    // ...
}
```

Error Handling

This step does not use `rename()`. After the new file name is recorded, the old file must be deleted. However, if `kv.meta.Set()` fails, the new file should not be deteted! Because an update failure doesn't guarantee that the state is unchanged, the state on disk may still points to the new file. **So if a state change fails, treat the state as unknown**. This is the same in distributed systems. If switching SSTables fails, keep using the old SSTables, but do not delete the new one.

Consider one more case. If the system recovers from an IO error (without restart) and can do `KV.Compact()` again, it must not break the SSTable left from the last failure. This is fine as long as each generated file name is unique. We use monotonic number to generate file names, so the number should increase on both success and failure.

Unused SSTables from IO errors or power loss can be deleted only if the disk state is known.

This discussion assumes the OS and file system handle IO errors correctly. In practice this may not hold. See this study: Can Applications Recover from fsync Failures?[2]

io.Closer

Since we opended multiple files, if `KV.Open()` returns an error, already opened parts should be closed. A small trick is to record opened parts in a slice.

```
type MultiClosers []io.Closer
type KV struct {
    // ...
    MultiClosers
}
```

Types with a `Close()` method match the `io.Closer` interface and can be added to this slice. You can add a `Close()` method to the slice, so there is no need to write a separate `KV.Close()`.

```
func (mc *MultiClosers) Close() (reterr error) {
    for _, item := range *mc {
        if err := item.Close(); err != nil {
            reterr = err
        }
    }
    *mc = nil
    return reterr
}
```

[2] https://www.usenix.org/system/files/atc20-rebello.pdf

0703: Multiple Levels

This step adds multiple SSTable levels:

```
type KV struct {
    // ...
    // main    SortedFile
     main    []SortedFile
}

type KVMetaData struct {
     Version uint64
    // SSTable string
     SSTables []string
}
```

The query used to have 2 levels. Now only small changes are needed:

```
func (kv *KV) Seek(key []byte) (SortedKVIter, error) {
     levels := MergedSortedKV{&kv.mem}
    for i := range kv.main {
         levels = append(levels, &kv.main[i])
     }
     iter, err := levels.Seek(key)
    // ...
}
```

The last level SSTable must record deleted keys. Modify the SSTable format as follows:

```
[ key length | val length | deleted | key data | val data ]
    4bytes       4bytes      1byte
```

```
type SortedArray struct {
     keys    [][]byte
     vals    [][]byte
     deleted []bool      // added
}
```

`KV.Compact()` converts the log to an SSTable and inserts it into **`KV.main[0]`**, instead of replacing the SSTable. Merging between SSTables will be implemented in the next step.

```
func (kv *KV) Compact() error
```

0704: Merge Levels

LSM-Tree Parameters

In the previous step, the log was turned into the top-level SSTable. Now implement the merge operation from 0700. Add 2 parameters:

```
type KVOptions struct {
    Dirpath string
    // LSM-Tree
    LogShreshold int
    GrowthFactor float32
}
```

- **`LogShreshold`** is the max key count in the log. When exceeded, convert to an SSTable.
- **`GrowthFactor`** is the size ratio (key count) between the next level and the current level.

We explained LSM-Tree using a **`GrowthFactor`** of 2. A larger ratio can also be used, trading more frequent merges for fewer levels. In an LSM-Tree, a key is written on average $O(\log N)$ times. This is called **write amplification** and can limit write throughput. **`GrowthFactor`** is the trade-off between query performance (level count) and write throughput.

B+Tree also has write amplification because it's a node-based data structure.

Merge SSTables

The **`KV.Compact()`** function uses these 2 parameters to decide when to merge SSTables:

```
func (kv *KV) Compact() error {
    if kv.mem.Size() >= kv.Options.LogShreshold {
        if err := kv.compactLog(); err !=nil {
            return err
        }
    }
    for i :=0; i < len(kv.main)-1; i++ {
        if kv.shouldMerge(i) {
            if err := kv.compactSSTable(i); err !=nil {
                return err
            }
            i--
            continue
        }
    }
    return nil
}
```

- **`KV.compactLog()`** is the previous step's **`KV.Compact()`**.
- **`KV.shouldMerge()`** decides whether to merge level **`i`** with **`i+1`** using exponential growth rule.
- **`KV.compactSSTable()`** merges 2 adjacent SSTables and replaces the originals.

Note that the last level must drop deleted keys. You can wrap the input iterator:

```
type NoDeletedSortedKV struct {
    SortedKV
}

func (kv NoDeletedSortedKV) Iter() (iter SortedKVIter, err error) {
    if iter, err = kv.SortedKV.Iter(); err !=nil {
        return nil, err
    }
    return NoDeletedIter{iter},nil
}
```

```
func (kv *KV) compactSSTable(level int) error {
    // ...
    file := SortedFile{FileName: filename}
    m := SortedKV(MergedSortedKV{&kv.main[level], &kv.main[level+1]})
    if len(kv.main) == level+2 {
        m = NoDeletedSortedKV{m}
    }
    err := file.CreateFromSorted(m)
    // ...
}
```

Trigger KV.Compact()

Updating the DB may create or merge SSTables. Although the **average** cost per update is $O(\log N)$, the **single** merge can take too long. There are 2 solutions:

1. Spread the work. Only merge part of the data each time and record progress.
2. Use a background thread to merge SSTables without blocking reads and writes.

Option 1 amortizes the merge across many updates and avoids high single-operation latency. Some hashtables, like Redis, use this idea to amortize rehash.

Option 2 involves concurrency. While replacing an SSTable, the app may still be using it. You must also limit write speed so it does not exceed merge speed. Most LSM-Tree DBs use this approach because they already support concurrent transactions.

To make an LSM-Tree based DB practical requires more work. So the code does not auto-trigger `KV.Compact()` yet. We'll wait until concurrency is implemented.

Split SSTables

Even with background merges, new and old files coexist during merging, and space usage can double in the worst case. In practice, a level is split into multiple SSTable files instead of one giant file. During merging, only one file, not a whole level, is merged into the next level. This reduces space use, and interruption does not lose all progress.

Since split boundaries are arbitrary, a file in the upper level may overlap with n files in the lower level. The merge result may also be split into m files.

0801: Indexes and KV

What Is an Index?

An index is extra information used to help search. Many books have an "Index" chapter. Examples include a book's table of contents, a library classification system, and search engines. All are indexes in a broad sense. One index matches one query pattern, so in OLTP you must build indexes ahead of time. Databases are not magic. They cannot know future queries patterns.

Relational DBs are made of rows and columns. OLAP databases are column-based. Our OLTP database is row-based. One row is one record. A query first finds the needed record, then reads fields from it. For now, one record is a KV pair. This K is the "primary key". It is like a page number in a book. V is like the page content. An index is extra KV data that leads to the primary key (page number), then the full record (page content).

Relational DBs use sorting data structures for indexes to support range queries. Some also implement hashtables, but that is niche.

KV for Indexes

Indexed data can be one or more columns (a tuple). Put indexed columns in K and the primary key in V. Example:

```
create table t (
    a string, b string, c string, d string,
    primary key a,
     index (b, c),
     index (c)
);
```

Type	Key	Value
Primary key	a	b, c, d
Index	b, c	a
Index	c	a

Indexes and primary keys are both KV sets. The difference is that the primary key is unique, so a record can be identified by its primary key. Indexed data does not need to be unique. But our KV store does not allow duplicate keys, so we use another plan: Add the primary key to K to make it unique, and leave V empty.

Type	Key	Value
Primary key	a	b, c, d
Index	b, c, a	empty
Index	c, a	empty

INDEX clause

We do not add new features in this step. Just add **`INDEX`** to **`CREATE TABLE`**.

```
type StmtCreatTable struct {
    table   string
    cols    []Column
    pkey    []string
    indices [][]string  // added
}
```

This struct must be converted to **Schema**, so update it too.

```
type Schema struct {
    Table   string
    Cols    []Column
    // PKey    []int
    Indices [][]int
}
```

Whether querying a primary key or an index, the first step is the same: query KV. So a primary key is just a special index. Thus we remove **PKey** from **Schema** and use **Indices[0]** instead. This will simplify some logic later.

Update **DB.execCreateTable()** to convert **StmtCreatTable** to **Schema**. Rules:

- Put the primary key in **Indices[0]**, other indexes after it.
- Add primary key columns to indexes.

For the example above, **Schema.Indices** is **[][]int{{0}, {1, 2, 0}, {2, 0}}**.

0802: Update Indexes

Follow the plan from the last step and update indexes when updating the DB. This step does not consider queries.

Key Encoding

Keys of different indexes must be distinct, so add the `indexNo` parameter and encode it into the key. `indexNo` is the position in `Schema.Indices` array (0 is the primary key).

```
func (row Row) EncodeKey(schema *Schema, indexNo int) []byte {
    key := append([]byte(schema.Table), 0x00, byte(indexNo))
    // ...
}

func (row Row) DecodeKey(schema *Schema, indexNo int, key []byte) error
func EncodeKeyPrefix(schema *Schema, indexNo int, prefix []Cell, pos bool) []byte
```

Indexed queries are not implemented yet, so set `indexNo` to 0 where it is used.

Maintain Indexes

Modify INSERT, UPDATE, and DELETE to update or remove index keys. Requirements:

- When adding a record, insert index keys.
- When deleting a record, delete index keys.
- When updating an existing record, the old index keys may need to be removed.

With indexes, a single update touches multiple keys. Atomicity across multiple keys will be handled later when transaction is implemented.

0803: Indexed Query

Range Query

We'll support indexed range queries. The **`IndexNo`** parameter is the position in **`Schema.Indices`** array (0 is the primary key).

```
type RangeReq struct {
    StartCmp ExprOp
    StopCmp  ExprOp
    Start    []Cell
    Stop     []Cell
    IndexNo  int        // added
}

type RowIterator struct {
    db      *DB         // added
    schema  *Schema
    indexNo int         // added
    iter    *RangedKVIter
    valid   bool
    row     Row
}
```

Modify **`decodeKVIter()`** to support iterating by index, with these rules:

- When **`indexNo == 0`**, the key is the primary key, same as before.
- When **`indexNo > 0`**, it's an index; extract the primary key and find the row via **`DB.Select()`**.

```
func (iter *RowIterator) Next() (err error) {
    if err = iter.iter.Next(); err !=nil {
        return err
    }
    iter.valid, err = iter.decodeKVIter()
    return err
}

func (iter *RowIterator) decodeKVIter() (valid bool, err error) {
    // ...
    if iter.indexNo >0 {
        // ...
    }else {
        // ...
    }
}
```

Index Selection

Modify **`matchRange()`** so that every index (including the primary key) is tried, and output a **`RangeReq`**. This means adding an **`indexNo`** parameter to the original logic and looping over it:

```
func matchRange(schema *Schema, cond interface{}) (*RangeReq, bool) {
    for indexNo := range schema.Indices {
        if req, ok := matchRangeByIndex(schema, indexNo, cond); ok {
            return req, ok
        }
    }
    return nil, false
}
```

With this, SQL supports range queries using indexes. However, there is a problem: when updating or deleting over a range, the old implementation may iterate and update KV at the same time. The iterator of **SortedArray** does not support this. Updating one row may insert or delete multiple keys, which breaks the iterator position. To fix this bug, first save the iteration results in memory, then execute the update.

```
func (db *DB) execUpdate(stmt *StmtUpdate) (count int, err error) {
    // ...
    oldRows := []Row{}
    for ; err == nil && iter.Valid(); err = iter.Next() {
        oldRows = append(oldRows, slices.Clone(iter.Row()))
    }
    if err != nil {
        return 0, err
    }
    for _, row := range oldRows {
        // ...
    }
    // ...
}
```

The iterator validity problem is common source of bug.

0804: Transaction Interface

Transactions and Atomicity

With indexes, one record in a table may use multiple keys. We need the transaction feature to ensure atomicity across multiple keys.

```
tx = kv.NewTX()
tx.Set("k1", "v1")
tx.Set("k2", "v2")
tx.Commit()
```

`NewTX()` enters a transaction. All KV operations (seek, get, set, del) should be moved to it. **`Commit()`** ends the transaction, writes the updates (commit), and guarantees atomicity.

Right now, log + checksum ensures single-key atomicity, If a log record contains multiple keys, we can get transaction atomicity. That can be done later. This step only adds interfaces and refactors code. Besides crash atomicity, we'll also handle concurrency later.

Transaction Updates

We can record updates inside a transaction and apply them together on commit.

```
type KVTX struct {
    target  *KV
    updates SortedArray
    levels  MergedSortedKV
}
```

- **`KVTX.updates`** stores updates in the transaction.
- **`KVTX.levels`** is used for reads inside the transaction.

Instead of updating **`KV.mem`**, we now update **`KVTX.updates`**, and postpone log writes.

```
func (tx *KVTX) SetEx(key []byte, val []byte, mode UpdateMode) (bool, error) {
    // ...
    if updated {
        if _, err = tx.updates.Set(key, val); err !=nil {
            return err
        }
    }
    return updated,nil
}
```

Transaction Reads

Reads in a transaction must see its own updates. This is just one more LSM-Tree level.

```
func (kv *KV) NewTX() *KVTX {
    tx := &KVTX{target: kv}
    tx.levels = MergedSortedKV{&tx.updates, &kv.mem}
    for i := range kv.main {
        tx.levels = append(tx.levels, &kv.main[i])
    }
    return tx
}
```

```
func (tx *KVTX) Seek(key []byte) (SortedKVIter, error) {
    iter, err := tx.levels.Seek(key)
    if err != nil {
        return nil, err
    }
    return filterDeleted(iter)
}
```

Ending a Transaction

Committing a transaction has 2 steps: write the log, update **`KV.mem`**. Same as before.

```
func (tx *KVTX) Commit() error { return tx.target.applyTX(tx) }

func (kv *KV) applyTX(tx *KVTX) error {
    if err := kv.updateLog(tx); err != nil {
        return err
    }
    kv.updateMem(tx)
    return nil
}
```

There should also be a **rollback** method to discard a transaction. For now, it does nothing.

```
func (tx *KVTX) Abort() {}
```

KV Interface

The old KV interface can stay, wrapped by transactions.

```
func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
    tx := kv.NewTX()
    defer tx.Abort()
    return tx.Get(key)
}
func (kv *KV) SetEx(key []byte, val []byte, mode UpdateMode) (bool, error) {
    tx := kv.NewTX()
    updated, err := tx.SetEx(key, val, mode)
    return abortOrCommit(tx, updated, err)
}
```

```
func abortOrCommit(tx *KVTX, updated bool, err error) (bool, error) {
    if err != nil {
        tx.Abort()
    } else {
        err = tx.Commit()
    }
    return err == nil && updated, err
}
```

DB Transactions

DB also gains a transaction interface, as a simple wrapper over **KVTX**.

```
type DB struct {
    KV KV
    // tables map[string]Schema
}

type DBTX struct {
    kv     *KVTX
    tables map[string]Schema
}

func (db *DB) NewTX() *DBTX {
    return &DBTX{kv: db.KV.NewTX(), tables: map[string]Schema{}}
}
func (tx *DBTX) Abort() { tx.kv.Abort() }
func (tx *DBTX) Commit() error { return tx.kv.Commit() }
```

```
type RowIterator struct {
    // db      *DB
    tx      *DBTX
    schema  *Schema
    indexNo int
    iter    *RangedKVIter
    valid   bool
    row     Row
}
```

With this, we'll get atomicity for multiple keys in indexes later, and also atomicity for multiple statements in one transaction.

0805: Transaction Atomicity

This step implements transaction atomicity. Consider power loss or errors returned midway. Concurrency is not considered yet.

Log Rollback

Currently, one KV update uses one log entry. If all updates in one transaction share one entry, atomicity can be achieved. This is one approach. Another is to **add a record type to mark the end of a transaction**.

```
const (
    EntryAdd    EntryOp = 0
    EntryDel    EntryOp = 1
    EntryCommit EntryOp = 2 // new
)

type Entry struct {
    key []byte
    val []byte
    // deleted bool
    op  EntryOp
}
```

- On database startup, only process records before **`EntryCommit`**.
- If writing the log returns an error midway, the next write must restore the file offset to the start of the transaction.

Do not blindly append to the file. Track the file offset and use **`WriteAt()`**. If **`Log.Write()`** returns an error, the file offset must not change. This is an old bug.

```
type Log struct {
    // ...
    writer  struct {
        offset    int64
        committed int64
    }
}

func (log *Log) Write(ent *Entry) error {
    data := ent.Encode()
    if _, err := log.fp.WriteAt(data, log.writer.offset); err !=nil {
        return err
    }
    log.writer.offset += int64(len(data))
    return nil
}
```

Unlike **`Write()`**, **`WriteAt()`** must return error if the number of written bytes is fewer than requested. This is stricter than **`Write()`** because it only deals with regular files.

Add **`Log.Commit()`** to write the new record type. Move **`fsync`** here as well.

```
func (log *Log) Commit() error {
    if err := log.Write(&Entry{op: EntryCommit}); err !=nil {
        return err
    }
    if err := log.fp.Sync(); err !=nil {
        return err
    }
    log.writer.committed = log.writer.offset
    return nil
}
func (log *Log) ResetTX() {
    log.writer.offset = log.writer.committed
}
```

Log.writer.committed saves the offset at the start of the transaction. If the TX is aborted, call **Log.ResetTX()** to reset **Log.writer.offset**. Using the new **Log** API, modify **KV.updateLog()**:

```
func (kv *KV) updateLog(tx *KVTX) error {
    defer kv.log.ResetTX()
    // TODO
    return kv.log.Commit()
}
```

These offsets must be maintained when reading the log on DB startup. Modify **DB.Read()**:

```
func (log *Log) Read(ent *Entry) (eof bool, err error)
```

Nested Transactions

A DB transaction can contain multiple statements, and each statement can update multiple keys. Not only must the whole transaction be atomic, each single statement must also be atomic. If an error occurs midway when an update statement is modifying multiple records, the statement must rollback. This requires nested transactions.

```
func (outer *DBTX) Update(schema *Schema, row Row) (updated bool, err error) {
    inner := outer.NewTX()
    updated, err = inner.update(schema, row, ModeUpdate)
    return abortOrCommit(inner, updated, err)
}
```

All **DBTX** APIs must be wrapped in a nested transaction. **DBTX** directly calls **KVTX**, which is where nested transaction is implemented.

```
func (tx *DBTX) NewTX() *DBTX {
    return &DBTX{kv: tx.kv.NewTX(), tables:map[string]Schema{}}
}
```

```
type KVTX struct {
    // target  *KV
    target interface{ applyTX(*KVTX) error }
    updates SortedArray
    levels MergedSortedKV
}
```

```
func (tx *KVTX) NewTX() *KVTX {
    inner := &KVTX{target: tx}
    inner.levels = slices.Concat(MergedSortedKV{&inner.updates}, tx.levels)
    return inner
}

func (tx *KVTX) Commit() error { return tx.target.applyTX(tx) }

func (tx *KVTX) applyTX(inner *KVTX) error
```

KVTX.target is changed to an interface to handle commit logic:

- For the outermost transaction, **target** is ***KV**.
- For nested transactions, **target** is the outer ***KVTX**.

For nested TXs, **applyTX()** simply moves **KVTX.updates** to the outer TX. Implement this.

0901: Snapshot Isolation

Isolation Levels

From now on, we consider concurrent transactions, meaning multiple transactions at the same time (nested transactions are not considered). One key question is: **can a transaction observe updates made by other concurrent transactions?** This is transaction **isolation**, the I in ACID. Ideally, a transaction should not observe updates from other transactions.

```
kv.Set("k1", "x")
tx1, tx2 = kv.NewTX(), kv.NewTX()
tx2.Set("k1", "y")
tx1.Get("k1")   // x
tx2.Get("k1")   // y
tx2.Commit()
kv.Get("k1")    // y
tx1.Get("k1")   // x
tx1.Abort()
```

Concurrency does not mean transactions must run in different threads. Even sequential code that uses multiple transactions counts as concurrent, as in the example above. Multi-threading will be considered later.

Not all databases implement this level of isolation. Many DBs implement lower isolation levels, allowing transactions to observe changes from others to some extent, which may cause unwanted **anomalies** for apps. Higher isolation levels may cost performance, so some DBs let you configure the isolation level to trade performance for anomalies.

There are many terms for anomalies, such as read committed, read uncommitted, repeatable read, phantom read, etc. These terms do not always have strict definitions. You must check the actual behavior of a database. Here, we aim for the highest isolation level and you can ignore these terms.

Locks

To reach the highest isolation level so that transactions do not observe each other, the simplest way is to disallow concurrency and allow only 1 transaction at a time. This is usually called **serializable** and can be implemented with locks (mutexes).

The lock granularity does not have to be the whole database. If 2 transactions access different keys, they can run at the same time if each key can be locked individually. These locks are internal database implementations, not just some simple mutexes. They can apply to a single key or a range.

Reducing lock granularity and using multiple locks can cause deadlocks when transactions take locks in different orders. The DB must detect deadlocks and fail a transaction, so the app can **retry** later.

Some databases let apps control locks, such as the `SELECT ... FOR UPDATE` feature, which can help avoid potential deadlocks.

Snapshot

There are other ways to reach the highest isolation level without locks, usually called **snapshot** isolation. One common approach is MVCC (Multi-Version Concurrency Control).

- In MVCC, each key has a monotonic version number.
- When a key is updated, a new version is added while keeping the old version.
- Queries in a transaction ignore versions created after the transaction starts.

The version number does not have to actually exist. If we ignore the MemTable, an LSM-Tree is a set of SSTables. Since SSTables are never modified, new key versions only appear in new SSTables. This set forms a snapshot, meaning a version of the whole DB. For a copy-on-write B+Tree, the root pointer is a snapshot.

The problem is that the MemTable is modified directly. To make sure a transaction only sees versions before its start time, you can use one of these options:

a. Copy the MemTable when the transaction starts.
b. Do not modify the MemTable on commit, but swap it with a new one.
c. Use a copy-on-write data structure for the MemTable so all snapshots can coexist.
d. Implement the MemTable using LSM-Tree ideas to get copy-on-write behavior.
e. Store a version with each key in the MemTable, and ignore new versions in a transaction.

Method a, b are the dumbest options, but since the MemTable size is limited, they do not affect overall time complexity. Choose one and modify `KV.NewTX()` and `KV.updateMem()`.

0902: Update Conflicts

Read-Modify-Update

The idea of a snapshot can prevent read anomalies in concurrent transactions, but updates still need care. Consider a simple counter:

```
tx = kv.NewTX()
a = tx.Get("a")
a = a + 1
tx.Set("a", a)
tx.Commit()
```

With concurrent transactions, both may read the same value and write the same result, so `a` increases only once. To prevent this is called **concurrency control** in databases.

Reading data first, then updating based on it, is called read-modify-update. The later update depends on the earlier read. To keep this **causality**, there are 2 approaches:

1. Use locks to ensure no other update happens between read and update.
2. Before update, check whether the earlier read has changed. If so, fail.

Option 1 locks the subject before using it, but may fail due to deadlocks (if multiple locks are involved). Option 2 checks causality before update and aborts if broken. Option 2 is called **optimistic concurrency control** (OCC), while locks are pessimistic.

Optimistic Concurrency Control

There are many ways to implement OCC. One way is to add a version number to each key:

- When reading a key, note its version number.
- Before updating (adding a new version), abort if a newer version exists.

Here we use another method: keep a history and, at commit time, check whether related keys were modified.

```
type KV struct {
    // ...
    snapshot uint64
    history  []UpdatedKey
    ongoing  []*KVTX
}
type UpdatedKey struct {
    snapshot uint64
    key      []byte
}

type KVTX struct {
    snapshot uint64
    // ...
}
```

`snapshot` increases on a successful commit. It acts like a timestamp in a broader sence.

```
func (kv *KV) applyTX(tx *KVTX) error {
    defer kv.untrackTX(tx)      // added
    if tx.updates.Size() ==0 {
        return nil
    }
    if kv.checkTXConflict(tx) {
        return ErrTXConflict    // added
    }
    if err := kv.updateLog(tx); err !=nil {
        return err
    }
    kv.updateMem(tx)
    kv.updateHistory(tx)        // added
    return nil
}

func (kv *KV) NewTX() *KVTX {
    tx := &KVTX{snapshot: kv.snapshot, target: kv}
    kv.ongoing =append(kv.ongoing, tx)
    // ...
}

func (kv *KV) updateHistory(tx *KVTX) {
    kv.snapshot++
    // TODO: append history
}
```

The commit step adds 3 functions:

- **`kv.untrackTX()`** maintains the history.
- **`kv.checkTXConflict()`** checks conflicts with history.
- **`kv.updateHistory()`** increments the timestamp and adds keys to history.

The last 2 are straightforward. **`kv.untrackTX()`** solves unbounded history growth:

- With no concurrent transactions, history can be discarded.
- Records preceding the earliest active transaction can be pruned.

Active transactions are kept in **`KV.ongoing`**, so **`kv.untrackTX()`** can drop old history.

```
func (kv *KV) untrackTX(tx *KVTX) {
    idx := slices.Index(kv.ongoing, tx)
    kv.ongoing = slices.Delete(kv.ongoing, idx, idx+1)
    // TODO: prune history
}
```

This design has a drawback: a long-running transaction can prevent history pruning.

0903: Multithread & Locks

Mutex

We'll consider multithreading in this step. These are our concerns:

1. When entering a transaction, we need to read fields from the `KV` struct, and when exiting a transaction, we need to modify `KV`, so we need a lock.
2. Appending the log must be ordered.
3. The MemTable and the log must stay consistent.

By protecting `NewTX()`, `Commit()`, and `Abort()` with a single lock, all of these are satisfied.

```
type KV struct {
    // ...
    mu sync.Mutex
}

func (kv *KV) NewTX() *KVTX {
    kv.mu.Lock()
    defer kv.mu.Unlock()
    // ...
}
func (kv *KV) applyTX(tx *KVTX) {
    kv.mu.Lock()
    defer kv.mu.Unlock()
    // ...
}
func (kv *KV) abortTX(tx *KVTX) {
    kv.mu.Lock()
    defer kv.mu.Unlock()
    // ...
}
```

In multithreaded programming, a "lock" is also called a mutex (**mut**ual **ex**clusion). It's used to prevent a code section from running concurrently.

With multithreading, transactions can run in parallel. Even if one transaction takes a long time, it does not block others. The parallel part is only between entering and exiting a transaction. The "enter" and "exit" steps themselves are serialized.

Reads and Writes

Often, reading is more important than writing. For a forum app, showing posts matters more than posting. We can treat **read-only** transactions and **read-write** transactions as separate concerns.

In the solution above, entering a transaction and committing a transaction share the same lock. The commit step includes disk writes and `fsync`. If the disk IO is saturated by heavy writes, this lock is held most of the time, making it hard for other read-only transactions to enter. We can use finer-grained locks so that the "enter" step is not affected by IO:

- Reduce the scope of `KV.mu` to only protect fields in `KV`, not IO.
- Add another lock: `KV.commit`, to serialize the commit step.

```
type KV struct {
    // ...
    mu     sync.Mutex
    commit sync.Mutex
}

func (kv *KV) applyTX(tx *KVTX) error {
    kv.commit.Lock()
    defer kv.commit.Unlock()
    defer kv.untrackTXSync(tx)

    if tx.updates.Size() == 0 {
        return nil
    }
    if kv.checkTXConflict(tx) {
        return ErrTXConflict
    }
    if err := kv.updateLog(tx); err != nil {
        return err
    }

    kv.mu.Lock()
    defer kv.mu.Unlock()
    kv.updateMem(tx)
    kv.updateHistory(tx)
    return nil
}

func (kv *KV) untrackTXSync(tx *KVTX) {
    kv.mu.Lock()
    defer kv.mu.Unlock()
    kv.untrackTX(tx)
}
```

This still satisfies concerns 1, 2, and 3. And read-only transactions can enter and exit freely without being blocked by busy read-write transactions. The commit step of read-write transactions is still serialized, though.

0904: Multithread & Channels

Auto-Compact

Earlier we mentioned that building SSTables and maintaining the LSM-Tree via **`KV.Compact()`** should be auto-triggered after updates and run in a background thread. This step does that, starting by revising lock usages:

- **`KV.compactLog()`** and **`KV.applyTX()`** both operate on the log and use the **`KV.commit`** lock.
- **`KV.compactLog()`** and **`KV.compactSSTable()`** modify **`KV.main`** and use the **`KV.mu`** lock.

```
func (kv *KV) compactLog() error {
    kv.commit.Lock()
    defer kv.commit.Unlock()
    // ...
    file := SortedFile{FileName: filename}
    // ...
    kv.mu.Lock()
    kv.main = slices.Insert(kv.main,0, file)
    kv.mem.Clear()
    kv.mu.Unlock()
    return kv.log.Truncate()
}

func (kv *KV) compactSSTable(level int) error {
    // ...
    file := SortedFile{FileName: filename}
    // ...
    kv.mu.Lock()
    kv.main = slices.Replace(kv.main, level, level+2, file)
    kv.mu.Unlock()
    // ...
    return nil
}
```

Go Channel

Add an option to control whether to auto-trigger **`KV.Compact()`** in a background thread:

```
type KVOptions struct {
    // ...
    AutoCompact bool
}

func (kv *KV) Open() (err error) {
    // ...
    if kv.Options.AutoCompact {
        kv.startCompactThread()
    }
    return nil
}
```

```
func (kv *KV) startCompactThread() {
    go func() {
        for {
            // TODO: call KV.Compact()
        }
    }()
}
```

This thread should not busy loop and waste CPU. It should be triggered after updates using a **channel**. A channel is a queue that passes data between threads and wakes consumers.

```
func producer(c chan Work) {
    for moreWork() {
        work := newWork()
        c <- work
    }
    close(c)
}
func consumer(c chan Work) {
    for {
        work, ok := <-c
        if !ok {
            break
        }
        workOn(work)
    }
}
func main() {
    c := make(chan Work, 0)
    go producer(c)
    go consumer(c)
}
```

- If the queue is empty, the receiver blocks until data arrives, without wasting CPU.
- If the queue is full, the sender blocks until data is consumed.

Use `make(chan T, size)` to create a channel. The queue size can be 0. Besides send and receive, a channel can also be `close`d **by the sender**.

- After `close`, the receiver returns immediately with ok=false, which can signal thread exit.
- After `close`, the sender must not send again, or it will panic.

This is similar to a Unix pipe: after the sender closes it, the receiver gets EOF.

A channel not initialized with `make()` is a nil channel. Send and receive on it block forever. It is rarely useful in practice.

Channels for Communication

Use a channel to notify a thread to run `KV.Compact()`. This channel does not need to carry data, only wake the receiver, so its type is an empty struct.

```
type KV struct {
    // ...
    updated chan struct{}
}
```

```
func (kv *KV) applyTX(tx *KVTX) error {
    kv.commit.Lock()
    defer kv.commit.Unlock()
    // ...
    if kv.Options.AutoCompact {
        kv.updated <- struct{}{}:
    }
    return nil
}

func (kv *KV) startCompactThread() {
    go func() {
        for {
            if _, ok := <-kv.updated; !ok {
                break
            }
            _ = kv.Compact()
        }
    }()
}

func (kv *KV) Close() error {
    close(kv.updated)
    // ...
}
```

The code looks simple, but it has bugs. It is a good example to learn multithreading.

Bug 1: a channel is a queue with limited capacity. When the queue is full, the sender blocks. This block is actually helpful, since it prevents the DB from ingesting data faster than it can merge SSTables, which would cause unbounded log growth. The queue size can use the **`LogShreshold`** option:

```
    kv.updated = make(chan struct{}, kv.Options.LogShreshold)
```

The problem is that **`KV.applyTX()`** blocks on send while holding a lock (**`KV.commit`**). And the receiver **`KV.Compact()`** also uses this lock. So the sender may block on the channel while the receiver blocks on the lock, causing a deadlock. The fix is to move the channel operation outside the lock scope.

Bug 2: when **`KV.Close()`** runs, **`KV.Compact()`** may still be running, and other threads may still be executing transactions. The database must wait for them to finish before closing. This is a bug from the last step. If **`KV.applyTX()`** sends after the channel is closed, it will **`panic`**. Two problems must be solved:

1. How to tell sender and receiver to exit? Can't **`close`** the channel as the sender will **`panic`**.
2. How to wait for threads to exit?

Notify Threads to Exit

To notify threads to exit, use another channel and **`select`** to operate on 2 channels:

```
type KV struct {
    // ...
    closing chan struct{}
}
```

```
func (kv *KV) applyTX(tx *KVTX) error {
    // ...
    if kv.Options.AutoCompact {
        select {
        case kv.updated <- struct{}{}:
        case <-kv.closing:
        }
    }
    return nil
}

func (kv *KV) Close() error {
    close(kv.closing)
    // ...
}
```

Go's **`select`** works on multiple channels. Whichever is ready first is executed. If all channels block, **`select`** blocks until one is ready.

This avoids sending to a closed channel. The receiver also use this for the exit signal:

```
func (kv *KV) startCompactThread() {
    go func() {
        for {
            ok := false
            select {
            case _, ok = <-kv.updated:
            case <-kv.closing:
            }
            if !ok {
                break
            }
            _ = kv.Compact()
        }
    }()
}
```

If **`<-kv.updated`** is executed, **`ok`** is always **`true`** because the channel is never closed.

Wait for Threads to Exit

A channel is a **synchronization primitive** and can solve any multithreading problem. Other primitives, like semaphores and condition variables, can also solve any problem. Learning multithreading means learning these tools.

For waiting for threads to exit, it is possible to use channels and a counter that is incremented and decremented on thread start and exit. The channels notified the change of the counter so you can wait for the counter to reach 0. This works but is not the simplest approach, because the standard library provides **`sync.WaitGroup`**.

```
type KV struct {
    // ...
    threads sync.WaitGroup
}
```

```go
func (kv *KV) NewTX() *KVTX {
    // ...
    kv.threads.Add(1)
    return tx
}

func (kv *KV) untrackTXSync(tx *KVTX) {
    // ...
    kv.threads.Add(-1)
}

func (kv *KV) startCompactThread() {
    kv.threads.Add(1)
    go func() {
        defer kv.threads.Add(-1)
        for {
            // ...
        }
    }()
}

func (kv *KV) Close() error {
    close(kv.closing)
    kv.threads.Wait()
    // ...
}
```

sync.WaitGroup is a counter. The **Wait()** method blocks until the count reaches 0.